Staff / Senior Machine Learning Engineer

Technical Interview Preparation Package — Xero, AI Products

System Design for ML • System Design for AI Agents (LLM / RAG)
45 questions • 5-step answer framework • whiteboard architectures
Tailored to the research→production boundary, distributed processing,
and real-time AI services for millions of small-business users

Prepared as an interview coach & Staff/Principal ML interviewer · covers MANGOS, mid-size & startup bar

How to Use This Package

> This interview is explicitly **not** about model internals or specific algorithms. It is about the **components, infrastructure and data** needed to run an ML / AI system in production, and the **interfaces between research and production**. Expect a problem statement, a whiteboard, and a request to draw the architecture. Every answer below follows the same five-step structure so you can drive the conversation rather than wait to be asked.

Step 1 — Clarify Requirements	Pin down scale (QPS, users, data volume), latency budget (real-time vs batch), freshness, accuracy/cost targets, and constraints. For Xero: millions of users, financial data, privacy, multi-tenant, AWS. Always restate assumptions out loud.
Step 2 — High-Level Design	Draw the boxes: data sources → ingestion → processing → store → train/serve → consumer. Name the research/production seam. Keep it to ~6–8 boxes.
Step 3 — Deep Dive	Go deep on the 2–3 components the interviewer cares about: feature store, serving, monitoring, the handoff harness. Show you know the failure modes.
Step 4 — Trade-offs	Every choice has a cost. Batch vs streaming, build vs buy, global vs per-tenant, fine-tune vs RAG. Naming trade-offs unprompted is the Staff signal.
Step 5 — Common Mistakes	What weaker candidates miss. Stating these shows you have operated systems in production, not just built prototypes.

What the Interviewer Is Really Testing (Senior vs Staff)

Both levels must produce a correct, production-ready architecture. The difference is scope of influence and the depth of trade-off reasoning. Calibrate your answers accordingly.

Senior ML Engineer	Owns a system end-to-end. Strong on production readiness, the research→production handoff, monitoring, and clean interfaces. Mentors juniors. Justifies the design for *this* problem with sound trade-offs.
Staff ML Engineer	Sets technical direction *across* teams. Designs the **platform**, not just the system — the reusable abstractions (feature store, serving layer, eval harness) other squads build on. Reasons about cost at fleet scale, technical-debt strategy, org/Conway implications, and how to bring people along.

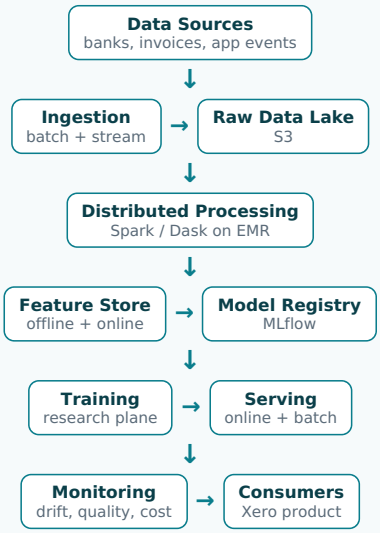
The Xero Lens — Bring These Into Every Answer

- **Research → production boundary.** Xero ML Engineers build the interfaces and harnesses that safely move Applied Scientists' models into production. Always name this seam: a model registry/contract, a reproducible packaging step, and an offline↔online parity guarantee.
- **Distributed processing.** Web-scale data via Spark / Dask on AWS EMR. Show you can push heavy feature engineering and batch scoring to a cluster, not a single box.
- **Orchestration.** Airflow / Prefect for pipeline scheduling, retries, backfills, dependencies and lineage.
- **ML tooling.** MLflow for experiment tracking + model registry; TensorFlow / PyTorch for models. Reference these by name when you describe the handoff and reproducibility.
- **Real-time at scale.** Services serving millions of users — think latency budgets, caching, autoscaling, graceful degradation, and cost per inference.
- **Financial domain.** Bank transactions, invoices, cash flow, accounting. Multi-tenant isolation, PII, auditability and explainability are first-class, not afterthoughts.

- **LLM features.** The next generation reduces toil and surfaces insights for small businesses — RAG over help/accounting content, natural-language querying of the books, and agentic bookkeeping assistance.
- **Leadership.** Mention how you would set the standard, document the interface, and unblock the team — the role is as much about lifting others as solving the problem yourself.

A Reusable Reference Architecture (Use as Your Default Skeleton)

When a question is open-ended, fall back to this skeleton and specialise it. It cleanly separates the research plane from the production plane — exactly what this interview probes.



Left/upper boxes = research & data plane; right/lower = production plane. The Model Registry is the contract between them.

Contents

Two sections. Section A covers classic ML system design; Section B covers AI-agent / LLM / RAG system design. Each question is a self-contained card with the five-step framework and a whiteboard-style architecture you can reproduce live.

Section A — ML System Design	M1. Design an ML Platform That Bridges Research and Production • M2. Design a Feature Store for a Fintech ML Platform • M3. Design a Real-Time Fraud / Anomaly Detection System for Bank Transactions • M4. Design an Invoice / Bank-Transaction Auto-Categorisation System at Scale • M5. Design a Cash-Flow Forecasting System for Small Businesses • M6. Design a Real-Time Model Serving Platform for Millions of Users • M7. Design a Distributed Training Pipeline on AWS EMR / Spark for Web-Scale Data • M8. Design a Model Monitoring and Drift-Detection System in Production • M9. Design an Automated (Continuous) Model Retraining Pipeline • M10. Design a Data Orchestration System for ML Pipelines (Airflow / Prefect) • M11. Design an Experiment Tracking and Model Registry System (MLflow) • M12. Design an Online Experimentation / A-B Testing Platform for ML Models • M13. Design a Batch Scoring System for Millions of Accounts • M14. Design a Recommendation System for Accounting Actions / Apps / Features • M15. Design a Late-Payment / Invoice-Payment Prediction System • M16. Design a Data Quality and Validation Framework for ML Pipelines • M17. Design a Feature Engineering Pipeline for Time-Series Financial Data • M18. Design an Entity Resolution / Deduplication System (Contacts, Vendors) • M19. Design an OCR + Document Understanding Pipeline for Receipts / Invoices • M20. Design a Customer Churn Prediction System for a SaaS Product • M21. Design a Multi-Tenant ML System with Data Isolation and Privacy • M22. Design a Cost-Optimisation Strategy for ML Training and Inference at Scale • M23. Design a Shadow Deployment and Canary Rollout System for ML Models • M24. Design a Human-in-the-Loop Labelling / Annotation Pipeline • M25. Design a Model Governance, Lineage and Reproducibility System
Section B — AI Agent / LLM / RAG	A1. Design a RAG System over Xero Help Docs and Accounting Knowledge • A2. Design a Natural-Language-to-Query (Text-to-SQL) Agent over Financial Data • A3. Design a Production RAG Ingestion and Indexing Pipeline • A4. Design an AI Agent That Automates Bookkeeping Tasks (Tool-Using Agent) • A5. Design an LLM Evaluation and Guardrails Framework • A6. Design an LLM Gateway / Inference Proxy (Routing, Caching, Fallback) • A7. Design a Multi-Agent Orchestration System for Complex Accounting Workflows • A8. Design a Hallucination Detection and Grounding / Citation System • A9. Design a Semantic Caching Layer for LLM Responses • A10. Design a Prompt Management and Versioning System • A11. Design a Vector Database / Embedding Store Architecture at Scale • A12. Design an LLM Observability and Tracing System • A13. Design a Fine-Tuning / Model-Customisation Pipeline (and When to Use It) • A14. Design a PII Redaction and Data-Privacy Layer for LLM Features • A15. Design an Agentic Invoice-Query Assistant with Safe Tool Calling • A16. Design a Conversational Assistant with Short-Term and Long-Term Memory • A17. Design a Document-Extraction Agent with Structured-Output Validation • A18. Design a Cost and Latency Optimisation Strategy for LLM Features • A19. Design an LLM Safety, Moderation and Jailbreak-Defence Pipeline • A20. Design an Offline-to-Online LLM Evaluation and Continuous-Improvement Loop

Section A — 25 ML System Design Questions

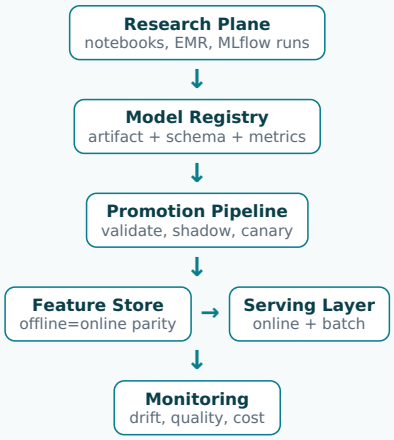
M1. Design an ML Platform That Bridges Research and Production

STEP 1 – CLARIFY REQUIREMENTS

This is the flagship Xero question. Clarify: who are the users (Applied Scientists in research, ML Engineers in production)? What model types (classification, forecasting, embeddings, LLMs)? What is the handoff cadence and who owns each side? Key non-functionals: reproducibility, offline/online feature parity, rollback, and audit lineage for financial data. I would confirm scale (dozens of models, millions of users), the cloud (AWS), and that we explicitly want to reduce toil in the research-to-production transition rather than rebuild models.

STEP 2 – HIGH-LEVEL DESIGN

The platform is split into a research plane and a production plane joined by a single contract: the model registry. Scientists train freely in notebooks/EMR; when ready they package a model artifact plus its feature schema and metadata into MLflow. A promotion pipeline validates the contract, runs shadow scoring, and only then deploys to the serving layer. A shared feature store guarantees the same feature logic in training and serving.



The registry is the only sanctioned interface between research and production; nothing reaches serving without passing through it.

STEP 3 – DEEP DIVE

The contract is the heart of the design. A registered model must declare its input feature schema, expected ranges, the feature-store keys it reads, the training data snapshot/commit, and evaluation metrics. The promotion pipeline enforces this with automated gates: schema validation, a parity check (re-score a sample offline and online, assert agreement), bias/regression checks against the current champion, and a shadow phase where the candidate scores live traffic without affecting users. Packaging is standardised (e.g. an MLflow pyfunc or container) so engineers never re-implement a scientist's preprocessing. Lineage ties every prediction back to a model version, feature version, and data snapshot.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Flexibility vs standardisation:** scientists want freedom; production wants contracts. Resolve with a thin, mandatory packaging interface and freedom inside it.
- **Build vs buy:** MLflow/SageMaker give you registry + tracking cheaply; a bespoke platform fits the org but is costly to maintain.
- **Shared feature logic vs duplication:** a feature store removes train/serve skew but adds a dependency every team must adopt.
- **Strict gates vs velocity:** heavy promotion gates reduce incidents but slow iteration; make gates fast and parallel.

STEP 5 – COMMON MISTAKES

- Treating the handoff as a code rewrite by engineers — the source of most train/serve skew and friction.
- No offline/online parity test, so the model behaves differently in production than in research.
- Skipping lineage, making a bad financial prediction impossible to audit or reproduce.
- Designing only for one model and not the platform abstractions other squads will reuse (a Staff-level miss).

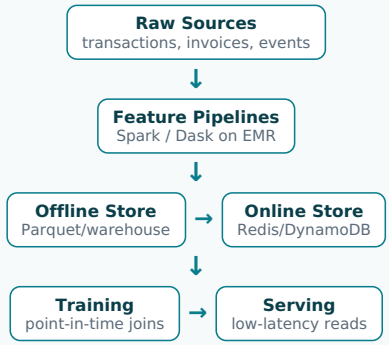
M2. Design a Feature Store for a Fintech ML Platform

STEP 1 – CLARIFY REQUIREMENTS

Clarify which features are needed online (low-latency serving) vs offline (training/batch), the freshness requirement per feature (real-time transaction signals vs daily aggregates), expected online QPS and p99 latency, point-in-time correctness needs for training, and multi-tenancy/PII constraints. Confirm scale: millions of entities (accounts, contacts, invoices), thousands of features, and that train/serve skew is the problem we are solving.

STEP 2 – HIGH-LEVEL DESIGN

Two stores backed by one definition. Feature pipelines (Spark/Dask on EMR) compute features and write them to an offline store (e.g. S3/Parquet or a warehouse) for training and a low-latency online store (e.g. Redis/DynamoDB) for serving. A shared transformation library ensures both paths use identical logic. A registry holds feature definitions, owners, and freshness SLAs.



Same transformation code feeds both stores; the registry tracks definitions and freshness.

STEP 3 – DEEP DIVE

Point-in-time correctness is the trickiest part: when building a training set, you must join each label with feature values as they were at that timestamp, not their current value, to avoid label leakage. Implement this with time-travel joins over the offline store. The online store is materialised by scheduled jobs (batch features) and streaming writers (real-time features) so serving reads are O(1) key lookups. Parity is enforced by computing both stores from one shared library and periodically reconciling samples. Backfills re-run the pipeline over history to populate new features.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Real-time vs batch features:** streaming gives freshness but costs more and is harder to keep consistent.
- **Online store choice:** Redis (fast, in-memory, costlier) vs DynamoDB (managed, scalable) vs in-process cache.
- **Pre-compute vs on-demand:** materialising everything is fast to serve but expensive; on-demand saves storage but adds latency.
- **Buy (Feast/Tecton) vs build:** faster start vs tighter fit to Xero's tenancy and lineage needs.

STEP 5 – COMMON MISTAKES

- Computing features differently in training and serving — the classic train/serve skew that silently degrades models.
- Ignoring point-in-time correctness and leaking future information into training labels.
- No freshness SLA or monitoring, so stale features quietly poison predictions.
- Putting PII in features without tenant isolation or masking.

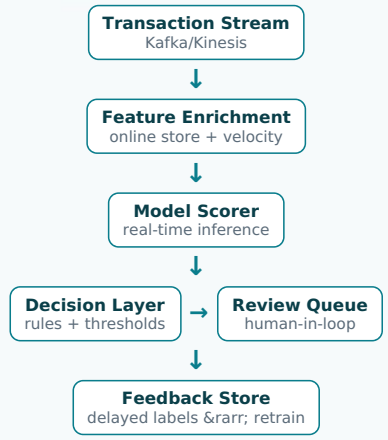
M3. Design a Real-Time Fraud / Anomaly Detection System for Bank Transactions

STEP 1 – CLARIFY REQUIREMENTS

Clarify the decision latency (must we block a transaction synchronously, or flag asynchronously?), volume (transactions/sec at peak), the cost asymmetry (a missed fraud vs a false decline), label availability and delay (chargebacks arrive weeks later), and whether we need explanations for the customer/regulator. Confirm multi-tenant scope and that we serve millions of small businesses.

STEP 2 – HIGH-LEVEL DESIGN

A streaming scorer enriches each transaction with features (velocity, deviation from the account's pattern) from the online feature store, runs a model, and emits a risk score. High scores route to a rules layer and human review queue; everything is logged for delayed labelling and retraining.



Synchronous path must meet a tight latency budget; labels return later and feed continuous retraining.

STEP 3 – DEEP DIVE

The hard problems are latency, imbalance, and label delay. Latency: features must be pre-materialised in the online store and the model kept small; aim for low tens of milliseconds end-to-end. Class imbalance (fraud is rare): use precision/recall and cost-weighted metrics, not accuracy, plus techniques like focal loss or careful thresholding calibrated to the false-decline budget. Label delay: maintain a feedback store that reconciles confirmed fraud/chargebacks back to the original transaction, then retrain regularly; in the meantime use proxy signals. A rules layer catches known patterns instantly and provides explainability that pure ML lacks.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Block synchronously vs flag asynchronously:** blocking stops fraud but risks false declines and latency on the payment path.
- **Rules vs ML:** rules are explainable and instant; ML catches novel fraud but is opaque — use both.

- **Threshold tuning:** high recall catches more fraud but annoys legitimate users; tie the threshold to business cost.
- **Model freshness vs stability:** frequent retraining adapts to new fraud but risks instability without strong validation.

STEP 5 – COMMON MISTAKES

- Optimising accuracy on an imbalanced problem instead of cost-weighted precision/recall.
- Ignoring label delay and assuming fresh ground truth is available.
- No human-review loop, so the model can never recover from systematic errors.
- Forgetting explainability, which fintech regulators and customers require.

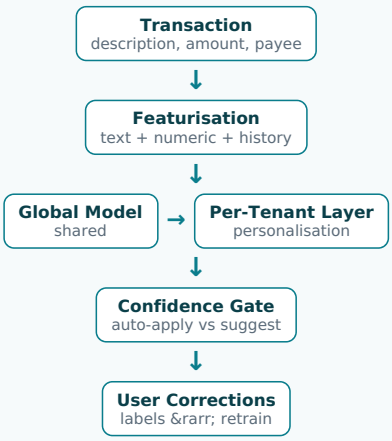
M4. Design an Invoice / Bank-Transaction Auto-Categorisation System at Scale

STEP 1 – CLARIFY REQUIREMENTS

Clarify whether categorisation is global, per-tenant, or hybrid (small businesses use custom chart-of-accounts categories). Clarify latency (real-time as a user codes a transaction vs nightly batch), the cold-start problem for new customers, the number of categories (can be large and tenant-specific), and the need for confidence so we only auto-apply when sure. Confirm millions of accounts and continuous user corrections as labels.

STEP 2 – HIGH-LEVEL DESIGN

A hybrid model: a global base model learns from all tenants' (privacy-safe) signals, while per-tenant personalisation adapts to each business's habits. Transactions are featurised (text of the description, amount, counterparty, history), scored, and either auto-applied above a confidence threshold or suggested for confirmation. User corrections stream back as fresh labels.



Confidence gating protects trust; corrections form a virtuous labelling loop.

STEP 3 – DEEP DIVE

Personalisation without leaking data across tenants is the core design choice. A global model captures broad patterns (a coffee-shop charge is usually 'meals'), while a lightweight per-tenant component (embeddings of the tenant's own historical mappings, or a few-shot lookup) captures idiosyncratic rules. Confidence calibration matters more than raw accuracy: auto-applying a wrong category erodes trust, so only auto-apply above a tuned threshold and otherwise suggest. The correction stream is gold-standard labelled data; pipe it into the feature store and retraining loop. For new tenants, fall back to the global model and rules until enough history accrues.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Global vs per-tenant models:** global generalises and is cheap to operate; per-tenant fits but explodes operational cost — hybrid balances both.
- **Auto-apply vs suggest:** auto-apply reduces toil but risks silent errors; suggest is safer but less magical.
- **Real-time vs batch scoring:** real-time feels instant; batch is cheaper for bulk imports.
- **Rich text models vs simple features:** transformers improve accuracy but raise latency and cost per inference.

STEP 5 – COMMON MISTAKES

- One global model with no personalisation, frustrating businesses with bespoke categories.
- Uncalibrated confidence, so auto-apply makes confident wrong decisions.
- Not capturing user corrections as labels — throwing away the best signal you have.
- Leaking one tenant's data into another's personalisation.

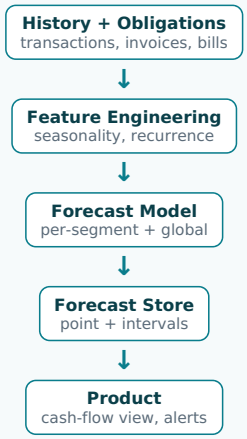
M5. Design a Cash-Flow Forecasting System for Small Businesses

STEP 1 – CLARIFY REQUIREMENTS

Clarify the forecast horizon (next 30/90 days), granularity (daily/weekly balance), whether we need point forecasts or full predictive intervals (uncertainty matters hugely for cash), the cold-start case for thin-history businesses, and refresh cadence (daily). Clarify inputs: historical transactions, recurring bills, outstanding invoices with due dates, and seasonality. Confirm millions of businesses with very different patterns.

STEP 2 – HIGH-LEVEL DESIGN

A batch forecasting pipeline pulls each business's transaction history and known future obligations, engineers time-series features, runs a forecasting model, and writes forecasts plus uncertainty bands to a store the product reads. Known future events (scheduled invoices/bills) are injected as covariates rather than predicted.



Deterministic future events are covariates; the model predicts the uncertain remainder.

STEP 3 – DEEP DIVE

Uncertainty is the product, not a footnote — a small business cares about the chance of going negative, so output quantiles/intervals, not just a mean. Combine a global model trained across all businesses (captures shared seasonality, helps thin-history accounts) with per-segment or hierarchical structure. Crucially, separate the deterministic component (a known invoice due on the 15th) from the stochastic component (irregular spend); inject the former as covariates so the model only forecasts genuine uncertainty. Run as a daily batch on EMR (millions of independent series parallelise perfectly). Backtest with rolling-origin evaluation and track calibration of the intervals, not just point error.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Global vs per-business models:** global handles cold start and scales; per-business fits but is operationally heavy — hierarchical/global-with-covariates wins.
- **Point forecast vs distribution:** intervals are more useful and more honest but harder to model and evaluate.
- **Classical (ETS/ARIMA) vs ML/DL:** classical is interpretable and robust on short series; ML captures cross-series patterns at higher cost.
- **Daily batch vs on-demand:** batch is cheap and predictable; on-demand suits what-if scenarios.

STEP 5 – COMMON MISTAKES

- Reporting only a point estimate, hiding the risk the customer actually needs.
- Forecasting events that are actually known (scheduled invoices), wasting model capacity and adding error.
- Evaluating with a random split instead of time-based backtesting, leaking the future.
- Ignoring calibration — intervals that are too narrow give false confidence about cash.

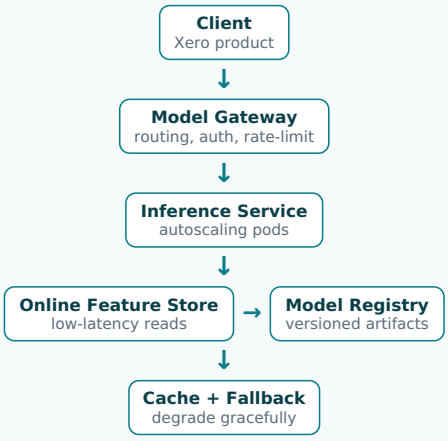
M6. Design a Real-Time Model Serving Platform for Millions of Users

STEP 1 – CLARIFY REQUIREMENTS

Clarify the latency budget (p50/p99), peak QPS, model sizes and frameworks (TF/PyTorch/sklearn), whether multiple models/versions co-exist, and freshness of features. Clarify availability target and graceful-degradation expectations (what happens if the model is down?). Confirm we want a shared platform many teams deploy onto, not one bespoke service.

STEP 2 – HIGH-LEVEL DESIGN

A model gateway fronts a fleet of autoscaling inference services. Requests carry an entity key; the service fetches online features, runs the model (loaded from the registry), and returns a prediction. A cache short-circuits repeat requests. Versioning, routing (canary/shadow), and a fallback path are first-class.



Gateway centralises routing/observability; each model is a versioned, independently scalable service.

STEP 3 – DEEP DIVE

Meeting p99 under load is the engineering core. Pre-fetch and co-locate features to avoid a slow remote hop; batch micro-requests where possible; keep models warm and right-sized; and use horizontal autoscaling keyed on latency and queue depth, not just CPU. The

gateway enables safe rollout: shadow a new version on live traffic, then canary a small percentage, then ramp. A fallback (cached prediction, simpler model, or default) preserves the user experience when the model fails or times out. Standardising packaging (containerised registry artifacts) lets any team deploy without bespoke infra — the platform mindset.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Real-time vs batch precompute:** precomputing predictions is cheapest and fastest to serve but only works for predictable inputs.
- **One big service vs per-model services:** isolation and independent scaling vs operational overhead.
- **Latency vs model complexity:** bigger models improve quality but blow the p99 budget — quantise/distil if needed.
- **Cache freshness vs hit rate:** longer TTL cuts cost but risks stale predictions.

STEP 5 – COMMON MISTAKES

- Designing for average latency and getting paged on p99 tail under load.
- No graceful degradation, so a model outage becomes a product outage.
- Fetching features over a slow path, dominating the latency budget.
- No canary/shadow, so a bad model version hits all users at once.

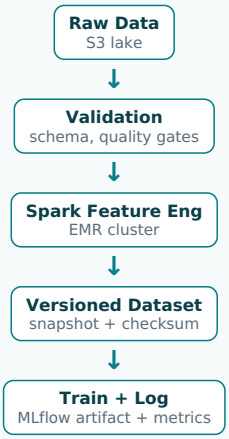
M7. Design a Distributed Training Pipeline on AWS EMR / Spark for Web-Scale Data

STEP 1 – CLARIFY REQUIREMENTS

Clarify data volume (terabytes of transactions), whether the bottleneck is data preparation (Spark-shaped) or model training (GPU-shaped), how often we retrain, reproducibility requirements, and cost ceiling. Clarify whether models are distributed-trainable (deep nets) or single-node after a big feature-engineering step. Confirm EMR/Spark is the mandated stack.

STEP 2 – HIGH-LEVEL DESIGN

An orchestrated pipeline: ingest and validate raw data, run distributed feature engineering on Spark/EMR, materialise a versioned training dataset, then train (single-node or distributed) with the artifact, metrics, and data snapshot logged to MLflow. Orchestrated by Airflow/Prefect with retries and backfills.



Orchestrated end-to-end (Airflow/Prefect); each run is reproducible from the dataset snapshot.

STEP 3 – DEEP DIVE

Most 'training at scale' is really data engineering at scale: the heavy lifting is distributed feature computation, so push it to Spark and keep the model-training step lean over the resulting dataset. Reproducibility requires snapshotting the exact training data (immutable, checksummed, versioned in S3) and pinning code/config so any run can be reproduced from the registry. Use transient/spot EMR clusters spun up per job and torn down to control cost. For genuinely distributed model training, use data-parallel frameworks; otherwise materialise features then train on a single right-sized box. Quality gates before training prevent wasting cluster hours on bad data.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Spot vs on-demand EMR:** spot is far cheaper but can be reclaimed — checkpoint to survive interruptions.
- **Distributed vs single-node training:** distribute only if the model genuinely needs it; coordination overhead is real.
- **Persistent vs transient clusters:** transient saves cost; persistent reduces startup latency for frequent jobs.
- **Recompute vs cache features:** caching engineered features speeds iteration at storage cost.

STEP 5 – COMMON MISTAKES

- Not snapshotting training data, so runs are irreproducible and audits fail.
- Distributing model training that would fit on one node, paying coordination overhead for nothing.
- Leaving large clusters running idle, burning budget.
- No data-quality gate, so the cluster trains happily on corrupt data.

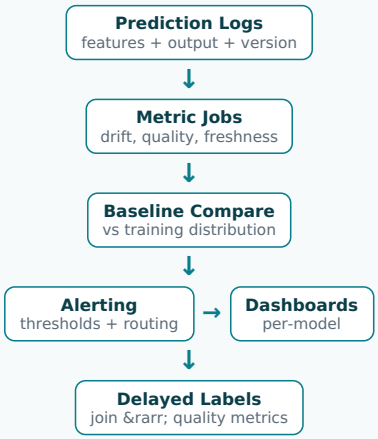
M8. Design a Model Monitoring and Drift-Detection System in Production

STEP 1 – CLARIFY REQUIREMENTS

Clarify what we monitor: operational health (latency, errors), data drift (input distribution shift), prediction drift (output shift), and model quality (needs labels, often delayed). Clarify label availability and delay, alerting thresholds and ownership, and whether monitoring is per-model or a shared platform. Confirm we serve financial predictions where silent degradation is costly.

STEP 2 – HIGH-LEVEL DESIGN

Every prediction is logged with its features, model version, and (eventually) its label. A monitoring service computes drift and quality metrics on a schedule, compares against a training-time baseline, and raises alerts that route to dashboards and on-call. Delayed labels join back to close the quality loop.



Input/prediction drift is available immediately; quality metrics wait for labels.

STEP 3 – DEEP DIVE

The key insight is that you usually cannot measure accuracy in real time because labels arrive late, so you rely on leading indicators: input drift (population stability index, KS/PSI per feature) and prediction drift signal trouble before quality metrics confirm it. Define the baseline from the training snapshot so 'drift' is always relative to what the model learned. Alerts must be actionable, not noisy — tune thresholds and aggregate to avoid alert fatigue. When labels arrive (chargebacks, user corrections, realised outcomes), join them back to compute true quality and trigger retraining. Make this a shared platform so every model is monitored by default, not per-team heroics.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Leading (drift) vs lagging (accuracy) signals:** drift is timely but noisy; accuracy is definitive but delayed.
- **Sensitive vs quiet alerting:** tight thresholds catch issues early but cause fatigue; loose ones miss slow rot.
- **Log everything vs sample:** full logging is complete but costly at scale; sampling saves money but can miss tail issues.
- **Per-model bespoke vs platform monitoring:** platform scales the org but must handle diverse model types.

STEP 5 – COMMON MISTAKES

- Monitoring only latency/errors and missing silent quality decay.
- No training baseline, so drift has nothing to compare against.
- Alert storms that get muted, so real incidents are ignored.
- Never closing the loop with delayed labels, so you never truly know accuracy.

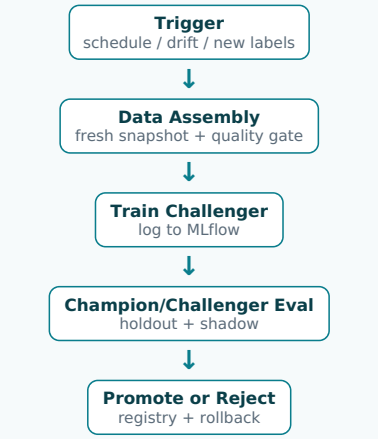
M9. Design an Automated (Continuous) Model Retraining Pipeline

STEP 1 – CLARIFY REQUIREMENTS

Clarify the retraining trigger (schedule, drift-based, or volume-of-new-labels), how a new model is validated before replacing the champion, rollback strategy, and guardrails against training on bad/poisoned data. Clarify cadence vs cost, and whether retraining is fully automated or requires human approval for financial models. Confirm reproducibility and audit needs.

STEP 2 – HIGH-LEVEL DESIGN

A trigger (cron, drift alert, or label threshold) kicks an orchestrated pipeline: assemble fresh training data, train a challenger, evaluate it against the champion on a holdout and via shadow scoring, and only promote if it wins on guarded metrics. Promotion goes through the registry with full lineage; rollback is one click.



No challenger replaces the champion without passing automated gates; every promotion is reversible.

STEP 3 – DEEP DIVE

Safe automation hinges on the champion/challenger gate. Never auto-deploy purely because a new model exists; require it to beat the current production model on a frozen evaluation set and in shadow, with checks for regressions on key segments and for fairness. Guard the input: a data-quality gate prevents retraining on corrupted or anomalous data (a poisoning vector). Keep humans in the loop for high-stakes financial models via an approval step, while low-risk models promote automatically. Every model carries lineage (data snapshot, code, metrics) so a regression can be diagnosed and rolled back instantly. Schedule on Airflow/Prefect with idempotent, retryable steps.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Frequent vs infrequent retraining:** frequent adapts fast but costs more and risks instability; infrequent is stable but stale.
- **Fully automated vs human-approved promotion:** automation scales; approval adds safety for regulated decisions.
- **Drift-triggered vs scheduled:** drift triggers are responsive but can thrash; schedules are predictable.
- **Aggressive vs conservative gates:** strict gates prevent bad deploys but can block genuine improvements.

STEP 5 – COMMON MISTAKES

- Auto-promoting any new model without beating the champion — 'newer' is not 'better'.
- No data-quality gate, allowing corrupted or adversarial data to retrain the model.
- No rollback path, so a bad automated deploy lingers in production.
- Evaluating only on aggregate metrics, masking regressions on important segments.

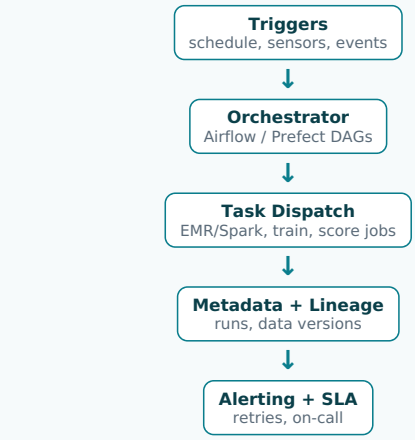
M10. Design a Data Orchestration System for ML Pipelines (Airflow / Prefect)

STEP 1 – CLARIFY REQUIREMENTS

Clarify the workloads (ingestion, feature jobs, training, batch scoring), their dependencies and SLAs, backfill needs, idempotency requirements, and how failures should be handled (retry, alert, skip). Clarify scale (hundreds of DAGs), multi-team ownership, and the need for lineage and observability. Confirm AWS/EMR integration and that this orchestrates the whole ML lifecycle.

STEP 2 – HIGH-LEVEL DESIGN

A scheduler (Airflow/Prefect) runs DAGs that express dependencies between tasks. Tasks dispatch heavy compute to EMR/Spark or training services rather than running in-process. A metadata layer records runs, data versions, and lineage; alerting and SLA monitors sit on top. Sensors gate downstream tasks on upstream data readiness.



Orchestrator coordinates; it does not do heavy compute itself — it dispatches to clusters.

STEP 3 – DEEP DIVE

Idempotency and backfills are what separate a robust pipeline from a fragile one. Every task should be safe to re-run and produce the same output for the same inputs, partitioned by date so backfills re-process specific windows without side effects. Use data-readiness sensors so a training job only starts once its upstream features land, rather than racing on a fixed clock. Keep the orchestrator thin: it schedules and tracks, but compute happens on EMR/Spark or dedicated services to avoid overloading workers. Capture lineage at the metadata layer so any artifact can be traced to the runs and data that produced it. Standardise DAG patterns so teams don't reinvent retries and alerting.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Airflow vs Prefect:** Airflow is mature and ubiquitous; Prefect offers a more Pythonic, dynamic model — pick per team familiarity.
- **Time-based vs sensor-based triggering:** schedules are simple; sensors are correct but add complexity.
- **Monolithic vs modular DAGs:** one big DAG is easy to see; modular DAGs are reusable but need cross-DAG coordination.
- **Centralised vs team-owned orchestration:** central governance vs team autonomy.

STEP 5 – COMMON MISTAKES

- Non-idempotent tasks that corrupt data on retry or backfill.
- Running heavy compute inside the orchestrator workers, causing instability.
- Time-based triggers that race ahead of late-arriving data.
- No lineage, so a downstream error can't be traced to its source.

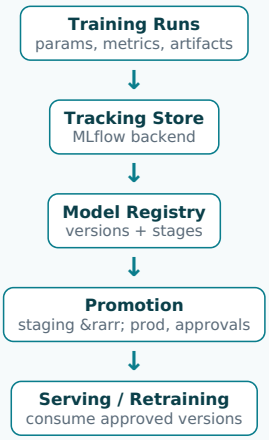
M11. Design an Experiment Tracking and Model Registry System (MLflow)

STEP 1 – CLARIFY REQUIREMENTS

Clarify what must be tracked (params, metrics, code version, data snapshot, artifacts), who consumes it (scientists comparing runs, engineers promoting models), the registry lifecycle stages (staging/production/archived), access control for financial models, and how this integrates with the serving and retraining pipelines. Confirm it is shared across many teams.

STEP 2 – HIGH-LEVEL DESIGN

An experiment-tracking service logs every training run with its full context. A model registry sits on top, holding versioned models with lifecycle stages and promotion transitions. Serving and retraining pipelines read approved versions from the registry; everything links back to runs for reproducibility.



The registry is the system of record for which model is live and why; serving pulls only from it.

STEP 3 – DEEP DIVE

The registry's value is governance and reproducibility, not just storage. Each model version links to the run that produced it, which links to the exact code commit, hyperparameters, metrics, and data snapshot — so any production prediction is fully auditable, which fintech demands. Lifecycle stages with controlled transitions (only a promotion pipeline or approver can move a model to production) prevent ad-hoc deploys. Integrate so the retraining pipeline registers challengers and the serving layer loads only the version tagged production, giving a single source of truth. Access control restricts who can promote financial models. Standardising this across teams stops every squad from inventing its own bookkeeping.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Managed MLflow vs self-hosted:** managed reduces ops; self-hosted gives control and data residency.
- **Log everything vs curated logging:** exhaustive logs aid reproducibility but bloat storage; curate the essentials.
- **Strict stage gates vs lightweight tagging:** gates add safety and friction; tags are flexible but less governed.
- **One central registry vs per-team:** central enables governance; per-team maximises autonomy.

STEP 5 – COMMON MISTAKES

- Tracking metrics but not the data snapshot/code version, breaking reproducibility.
- Letting engineers deploy models that bypass the registry, losing the source of truth.
- No access control, so anyone can promote a model to production.
- Treating it as a passive log instead of wiring it into serving and retraining.

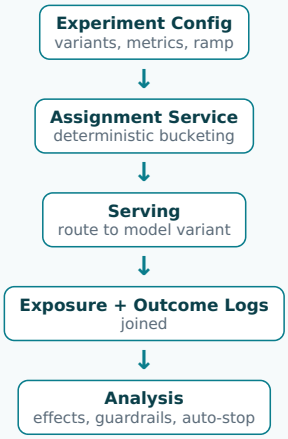
M12. Design an Online Experimentation / A-B Testing Platform for ML Models

STEP 1 – CLARIFY REQUIREMENTS

Clarify the unit of randomisation (user, tenant, transaction), the metrics (business KPIs plus model metrics), guardrail metrics to protect (latency, error rate, revenue), minimum detectable effect and required sample size, and how to avoid interference between concurrent experiments. Confirm the need for safe rollout and that financial outcomes are part of the metric set.

STEP 2 – HIGH-LEVEL DESIGN

An assignment service deterministically buckets units into variants and logs exposure. The serving layer routes each unit to its model variant. An analysis pipeline joins exposures with outcomes, computes effects with significance and guardrails, and surfaces results in a dashboard with automatic ramp/stop controls.



Assignment is deterministic and logged; analysis ties exposure to downstream financial outcomes.

STEP 3 – DEEP DIVE

Trustworthy experimentation depends on correct assignment and honest analysis. Bucketing must be deterministic and sticky (same unit always sees the same variant via hashing) and logged at exposure time so analysis only counts units that actually saw the treatment. Choose the randomisation unit to avoid leakage: if predictions affect a whole tenant, randomise by tenant, not transaction. Pre-register the primary metric and minimum detectable effect to avoid p-hacking, and watch guardrail metrics (latency, error rate, declines) to auto-stop a harmful variant. Account for multiple concurrent experiments via layered/orthogonal assignment so they don't confound. For long-horizon financial outcomes, use surrogate metrics with care.

STEP 4 – TRADE-OFFS TO DISCUSS

- **User vs tenant randomisation:** user gives more samples; tenant prevents within-business interference.
- **Short surrogate metrics vs true long-term outcomes:** surrogates are fast but can mislead.
- **Fixed-horizon vs sequential testing:** sequential allows early stopping but needs careful statistics.
- **Many concurrent experiments vs isolation:** throughput vs risk of confounding.

STEP 5 – COMMON MISTAKES

- Non-deterministic or unlogged assignment, corrupting the analysis.
- Randomising at the wrong unit and leaking treatment across the control group.
- Peeking at results and stopping early without sequential correction.
- Ignoring guardrail metrics, so a variant that lifts one KPI quietly harms another.

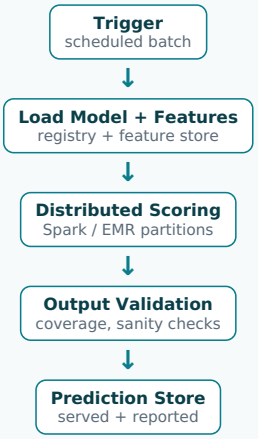
M13. Design a Batch Scoring System for Millions of Accounts

STEP 1 – CLARIFY REQUIREMENTS

Clarify the cadence (nightly/weekly), how predictions are consumed (precompute for fast serving, trigger alerts, feed reports), the freshness tolerance, the cost ceiling, and how to handle partial failures and backfills. Clarify that scoring must be reproducible and traceable to a model version. Confirm distributed compute (Spark/EMR) is expected for the volume.

STEP 2 – HIGH-LEVEL DESIGN

An orchestrated batch job loads the relevant model version, reads features for all entities, scores them in parallel on Spark/EMR, and writes predictions (with model version) to a store that serving and the product read. Quality checks validate the output before publishing.



Precomputed predictions let the online path be a cheap key lookup; outputs are validated before publishing.

STEP 3 – DEEP DIVE

Batch scoring trades freshness for huge cost efficiency and simplicity: instead of running a model on every request, you precompute predictions for all entities and serve them as fast lookups. Partition the work so millions of independent rows score in parallel across the cluster, and make the job idempotent and restartable so a failed partition can be retried without double-publishing. Validate the output before it goes live: check coverage (did we score everyone we expected?), distribution sanity (no sudden collapse), and tag every

prediction with the model version for traceability. Atomic publish (write-then-swap) avoids serving a half-written batch. Backfills re-score historical windows when a model changes.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Batch vs real-time scoring:** batch is far cheaper and simpler but predictions can be stale.
- **Score-everyone vs score-on-demand:** full coverage simplifies serving but wastes compute on inactive accounts.
- **Frequency vs cost:** more frequent batches are fresher but pricier.
- **Overwrite vs append predictions:** overwrite is simple; append preserves history for audit.

STEP 5 – COMMON MISTAKES

- Publishing a partially-completed batch and serving inconsistent predictions.
- No output validation, so a broken run silently ships bad predictions to millions.
- Not tagging predictions with model version, breaking traceability.
- Non-restartable jobs, forcing a full re-run on any failure.

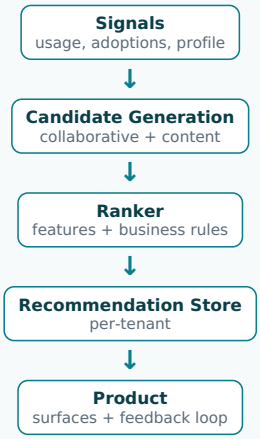
M14. Design a Recommendation System for Accounting Actions / Apps / Features

STEP 1 – CLARIFY REQUIREMENTS

Clarify what we recommend (next best action, marketplace apps, unused features), the objective (engagement, task completion, retention), the feedback signal (clicks, adoptions), cold-start for new businesses, and whether recommendations are real-time or batch. Clarify diversity/relevance balance and that recommendations must be appropriate for a financial product. Confirm millions of tenants with sparse interactions.

STEP 2 – HIGH-LEVEL DESIGN

A classic two-stage recommender: a candidate generator narrows the catalogue using collaborative and content signals, then a ranker scores candidates with rich features and business rules. Recommendations are precomputed in batch and refreshed, with a real-time re-rank layer for context.



Two-stage retrieve-then-rank keeps it scalable; feedback closes the learning loop.

STEP 3 – DEEP DIVE

Cold start and feedback loops dominate. New businesses have no interaction history, so lean on content-based signals (industry, size, features used) and popular defaults until behaviour accrues. The two-stage pattern keeps it tractable at catalogue scale: cheap candidate generation reduces millions of items to hundreds, then an expensive ranker orders them. Guard against feedback loops where the model only ever recommends what it already surfaced — inject exploration. Apply business rules as a final layer (don't recommend an app a tenant already has). Evaluate offline with ranking metrics but validate online with A/B tests, since offline metrics rarely predict engagement.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Collaborative vs content-based:** collaborative is powerful with data but fails cold-start; content works immediately but is narrower — hybridise.
- **Batch vs real-time recommendations:** batch scales cheaply; real-time captures session context.
- **Relevance vs diversity/exploration:** pure relevance creates filter bubbles and starves learning.
- **Offline metrics vs online lift:** offline ranking metrics are convenient but weakly correlated with real engagement.

STEP 5 – COMMON MISTAKES

- Ignoring cold-start and showing nothing useful to new businesses.
- No exploration, so the model reinforces its own past recommendations.
- Optimising offline metrics that don't translate to online engagement.
- Skipping business rules and recommending irrelevant or already-owned items.

M15. Design a Late-Payment / Invoice-Payment Prediction System

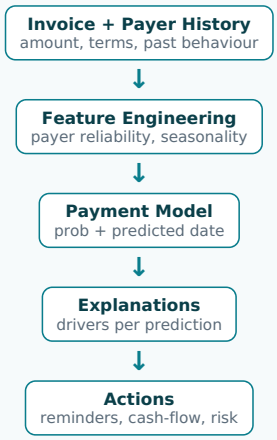
STEP 1 – CLARIFY REQUIREMENTS

Clarify the prediction target (will this invoice be paid late? by how many days? probability of default?), horizon, how the prediction is

used (chase reminders, cash-flow input, credit risk), label definition and delay, and fairness/explainability since it touches customers' payers. Clarify per-invoice vs per-payer modelling. Confirm millions of invoices across diverse industries.

STEP 2 – HIGH-LEVEL DESIGN

A batch (with optional real-time) classifier scores each outstanding invoice using invoice attributes, payer history, and the business's collection behaviour, outputting a probability and predicted pay date. Outputs drive reminders, cash-flow forecasts, and risk flags, with explanations attached.



Payer-level features carry most signal; explanations make outputs actionable and defensible.

STEP 3 – DEEP DIVE

The signal lives in payer behaviour, so the key feature engineering is aggregating each payer's historical punctuality across (privacy-permitting) the network, plus the issuing business's own terms and follow-up patterns. Treat label delay carefully: an invoice's outcome is only known after its due date passes, so use point-in-time features and time-based validation. Because outputs can influence how a business treats a customer, attach explanations (top contributing features) and avoid using sensitive proxies. Calibrate probabilities so '70% likely late' means what it says, since downstream cash-flow forecasting consumes them. Run mainly as batch with refresh on new events.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Classification vs regression of days-late:** probability is simpler and calibratable; days-late is richer but noisier.
- **Per-payer vs per-invoice modelling:** payer-level captures behaviour; invoice-level captures specifics — combine.
- **Network features vs strict isolation:** cross-tenant payer signals boost accuracy but raise privacy concerns.
- **Accuracy vs explainability:** complex models predict better but are harder to justify to customers.

STEP 5 – COMMON MISTAKES

- Leaking post-due-date information into features and inflating offline accuracy.
- Shipping uncalibrated probabilities that mislead cash-flow forecasting.
- No explanation, leaving the business unable to act on or trust a flag.
- Using sensitive payer attributes that create fairness or compliance risk.

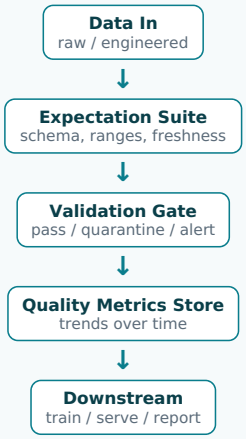
M16. Design a Data Quality and Validation Framework for ML Pipelines

STEP 1 – CLARIFY REQUIREMENTS

Clarify what 'quality' means here: schema conformance, null/range checks, distribution stability, referential integrity, and freshness. Clarify where checks run (ingestion, pre-training, pre-serving), what happens on failure (block, quarantine, alert), and who owns thresholds. Confirm this is a shared, reusable framework across pipelines, not ad-hoc asserts, and that financial data correctness is critical.

STEP 2 – HIGH-LEVEL DESIGN

A validation library defines declarative expectations per dataset. Checks run as gates at pipeline boundaries: ingestion, before training, before serving. Failures quarantine bad data and alert owners; results and metrics are logged for trend analysis and feed monitoring.



Checks are gates, not afterthoughts; bad data is quarantined before it reaches models.

STEP 3 – DEEP DIVE

The framework must be declarative and reusable so teams add checks without bespoke code — expectations live with the dataset (e.g. a Great-Expectations-style suite). Position gates at boundaries: validate on ingestion to catch upstream breakage, before training to avoid learning from corrupt data (a poisoning defence), and before serving to avoid scoring on malformed features. Distinguish hard failures (block the pipeline) from soft ones (warn and continue), and quarantine rather than silently drop bad records so they can be investigated. Track quality metrics over time so gradual drift in data quality is visible, and connect to the same alerting used for model monitoring. The hardest part is setting thresholds that catch real problems without crying wolf.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Strict vs lenient gates:** strict blocks bad data but can halt pipelines on benign anomalies; lenient keeps flowing but lets issues through.
- **Declarative suites vs custom code:** declarative scales across teams; custom handles edge cases.
- **Block vs quarantine vs alert-only:** trade safety for availability per dataset criticality.
- **Static thresholds vs learned baselines:** learned adapts but can normalise creeping degradation.

STEP 5 – COMMON MISTAKES

- Validating only schema and missing distribution drift in valid-but-wrong data.
- Failing open — logging a warning but training/serving on bad data anyway.
- Thresholds so noisy that teams disable the checks.
- Checking at one point only, so issues introduced mid-pipeline slip through.

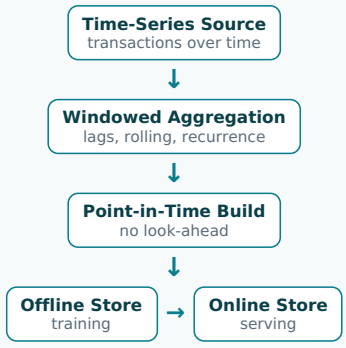
M17. Design a Feature Engineering Pipeline for Time-Series Financial Data

STEP 1 – CLARIFY REQUIREMENTS

Clarify the features needed (rolling aggregates, lags, recurrence detection, seasonality, trend) and over what windows, the freshness requirement (intraday vs daily), point-in-time correctness for training, and how features serve both batch and real-time. Clarify volume (millions of accounts, long histories) and that look-ahead leakage must be prevented. Confirm Spark/EMR for the heavy aggregation.

STEP 2 – HIGH-LEVEL DESIGN

A distributed pipeline computes windowed time-series features (lags, rolling stats, recurrence, calendar effects) per entity on Spark/EMR, writing point-in-time-correct features to the offline store and materialising the latest values to the online store. Shared transformation code keeps both consistent.



Window functions on Spark scale across millions of series; the same code feeds offline and online.

STEP 3 – DEEP DIVE

Avoiding temporal leakage is the central discipline: every rolling/lag feature for a timestamp t must use only data strictly before t, which Spark window functions handle if framed correctly, and training joins must be point-in-time so labels aren't paired with future-derived features. Recurrence detection (spotting monthly subscriptions or rent) is high-value and best computed as a feature rather than learned implicitly. Calendar and seasonality features (day-of-week, month-end, fiscal periods) encode known structure cheaply. Partition by entity so millions of independent series compute in parallel, and incrementally update rather than recomputing full history each run. The same transformation library must produce online features to avoid skew.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Incremental vs full recompute:** incremental is cheap but stateful and trickier; full recompute is simple but expensive.
- **Many windows vs few:** more windows add signal but inflate cost and dimensionality.
- **Precompute vs on-demand:** precomputing rolling stats speeds serving; on-demand saves storage.
- **Streaming freshness vs batch simplicity:** intraday features cost more and complicate consistency.

STEP 5 – COMMON MISTAKES

- Look-ahead leakage from incorrectly framed windows, inflating offline metrics.
- Different window logic in training vs serving, causing skew.
- Recomputing entire histories nightly when incremental updates suffice.
- Ignoring point-in-time joins and pairing labels with future information.

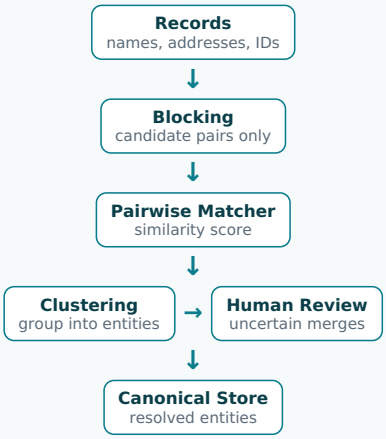
M18. Design an Entity Resolution / Deduplication System (Contacts, Vendors)

STEP 1 – CLARIFY REQUIREMENTS

Clarify the entities (vendors, customers, contacts) and what counts as a match (same legal entity despite spelling/format differences), the precision/recall trade-off (a wrong merge corrupts financial records), whether matching is within-tenant or cross-tenant, real-time vs batch, and how humans confirm uncertain merges. Confirm scale (millions of records) makes all-pairs comparison infeasible.

STEP 2 – HIGH-LEVEL DESIGN

A blocking step generates candidate pairs cheaply (so we don't compare all pairs), a matching model scores each pair's similarity, a clustering step groups matches into entities, and uncertain merges go to human review. Confirmed merges update a canonical entity store.



Blocking makes the problem tractable; humans confirm high-impact merges to protect financial integrity.

STEP 3 – DEEP DIVE

Scale forces blocking: comparing every record to every other is quadratic and impossible at millions, so we generate candidate pairs via cheap keys (normalised name tokens, postcode, fuzzy hashes) and only score those. The matcher combines string similarity, structured-field agreement, and learned weights into a match probability. Because a false merge corrupts a customer's books, bias toward precision and route borderline pairs to human review rather than auto-merging. Clustering must be consistent (transitive closure with conflict resolution) so A=B and B=C imply one entity. Keep merges reversible and audited. Run as batch with incremental matching for new records in near-real-time.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Precision vs recall:** false merges corrupt data (favour precision); missed merges leave duplicates — tune to cost.
- **Aggressive vs conservative blocking:** aggressive blocking is cheap but misses true pairs; loose blocking is thorough but costly.
- **Auto-merge vs human-confirm:** automation scales; review protects high-impact records.
- **Rules vs learned matcher:** rules are transparent; learned handles messy variation.

STEP 5 – COMMON MISTAKES

- Attempting all-pairs comparison and failing to scale.
- Auto-merging on weak evidence and corrupting customers' financial records irreversibly.
- Inconsistent clustering that creates contradictory entity groupings.
- No audit trail or undo for merges.

M19. Design an OCR + Document Understanding Pipeline for Receipts / Invoices

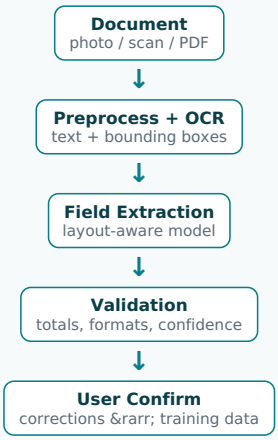
STEP 1 – CLARIFY REQUIREMENTS

Clarify input variety (photos, scans, PDFs, many languages and layouts), the fields to extract (vendor, date, total, tax, line items), accuracy targets and confidence handling, latency (real-time on upload vs batch), and how low-confidence extractions are corrected by users. Confirm millions of documents and that extracted values feed accounting records, so errors are costly.

STEP 2 – HIGH-LEVEL DESIGN

A staged pipeline: preprocess and OCR the image to text+layout, a document-understanding model extracts structured fields using text

and spatial features, a validation layer checks consistency (totals add up), and low-confidence fields route to user confirmation. Corrections become training data.



Spatial layout matters as much as text; validation and confidence gating protect the ledger.

STEP 3 – DEEP DIVE

Layout is signal: where a number sits (near 'Total', bottom-right) disambiguates it, so use layout-aware extraction rather than treating OCR output as a flat string. Confidence handling is the product-defining choice — auto-fill high-confidence fields, but surface low-confidence ones for quick user confirmation rather than silently guessing, since a wrong total corrupts the books. Validation rules (line items sum to subtotal, tax matches rate, date is plausible) catch extraction errors cheaply. The correction loop is invaluable labelled data; capture it. Handle the long tail of formats and languages by combining a general model with templates for high-volume vendors. Run real-time on upload with a batch path for bulk imports.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Off-the-shelf OCR/IDP vs custom models:** managed services are fast to adopt; custom fits Xero's documents and cost.
- **Auto-fill vs confirm:** auto-fill is magical but risks silent errors; confirmation is safer.
- **General model vs per-vendor templates:** templates are accurate for common vendors; general handles the tail.
- **Real-time vs batch:** real-time delights on upload; batch is cheaper for bulk.

STEP 5 – COMMON MISTAKES

- Treating OCR text as flat and discarding layout, hurting field extraction.
- Auto-applying low-confidence extractions and corrupting financial records.
- Skipping arithmetic validation that would catch obvious errors.
- Not capturing user corrections to improve the model.

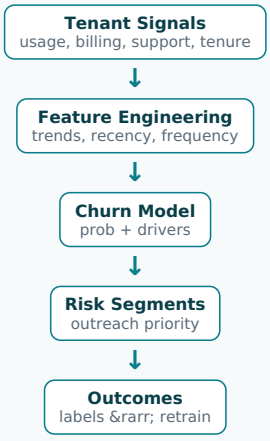
M20. Design a Customer Churn Prediction System for a SaaS Product

STEP 1 – CLARIFY REQUIREMENTS

Clarify the churn definition (subscription cancellation, inactivity, downgrade) and horizon (30/90 days), how predictions are used (retention outreach, in-product nudges), the label delay, and the need for explanations so retention teams can act. Clarify per-tenant scope and class imbalance. Confirm the goal is actionable, calibrated risk plus drivers, not just a score.

STEP 2 – HIGH-LEVEL DESIGN

A batch classifier scores active tenants on engagement, billing, support, and product-usage features, outputting churn probability plus top drivers. High-risk tenants flow to retention workflows; outcomes feed back as labels for retraining.



Drivers turn a score into action; the label loop closes only after the churn horizon elapses.

STEP 3 – DEEP DIVE

Actionability beats raw accuracy: a churn score is useless unless the retention team knows why and can intervene, so output the top contributing factors per tenant (declining logins, failed payments, rising support tickets). Trend features (is engagement falling?) outperform static snapshots. Class imbalance and delayed labels require time-based validation and cost-aware thresholds tuned to the

value of a retained customer versus outreach cost. Beware the intervention feedback loop: if retention acts on predictions, the realised churn no longer reflects the untreated model, complicating evaluation — use holdout groups. Run as batch (weekly) since churn evolves slowly. Calibrate probabilities so 'high risk' is prioritised sensibly.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Lead time vs accuracy:** predicting churn earlier gives time to act but is harder and less precise.
- **Accuracy vs explainability:** complex models score better but give weaker drivers for outreach.
- **Aggressive vs targeted outreach:** contacting more at-risk tenants saves some but annoys others and costs more.
- **Measuring impact:** acting on predictions contaminates labels unless you keep a control holdout.

STEP 5 – COMMON MISTAKES

- Delivering a score with no drivers, leaving retention unable to act.
- Random train/test splits that leak future behaviour into the past.
- Ignoring the intervention loop and mis-measuring the model after retention acts.
- Optimising accuracy on an imbalanced target instead of business-cost-weighted metrics.

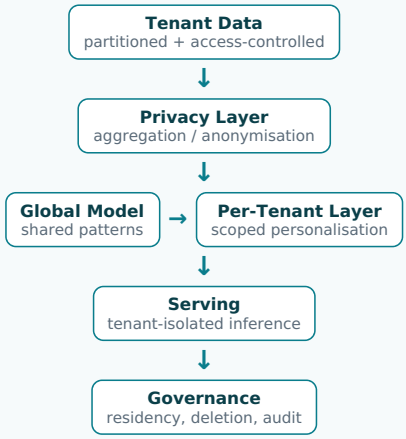
M21. Design a Multi-Tenant ML System with Data Isolation and Privacy

STEP 1 – CLARIFY REQUIREMENTS

Clarify the isolation requirement (logical vs physical separation), whether models are global, per-tenant, or hybrid, what cross-tenant learning is permitted (aggregate patterns yes, raw data no), regulatory constraints (data residency, PII), and how a tenant's deletion request propagates. Confirm millions of tenants makes per-tenant models operationally heavy and that trust is paramount in a financial product.

STEP 2 – HIGH-LEVEL DESIGN

A shared global model trains on privacy-safe aggregated signals, never raw cross-tenant data; tenant-specific personalisation stays scoped to each tenant. Feature pipelines and stores are tenant-partitioned with access controls; data residency and deletion are enforced at the storage layer.



Tenants benefit from shared learning without their raw data ever crossing the boundary.

STEP 3 – DEEP DIVE

The design must let tenants benefit from collective intelligence without exposing each other's data. Train global models on aggregated or anonymised signals so no tenant's raw records leak, while keeping any personalisation strictly tenant-scoped. Enforce isolation in infrastructure: partition feature stores by tenant, apply row/namespace-level access controls, and tag all data so a deletion request can purge a tenant everywhere (including retraining sets) to satisfy 'right to be forgotten'. Respect data residency by region-pinning storage and compute. Per-tenant models give the best fit but explode operational cost at millions of tenants, so prefer a global-plus-personalisation hybrid. Audit every access for compliance.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Global vs per-tenant models:** global scales and shares learning; per-tenant fits but is operationally infeasible at scale.
- **Logical vs physical isolation:** logical is cheaper; physical is stronger and sometimes legally required.
- **Cross-tenant learning vs strict isolation:** aggregate learning improves models but must be provably privacy-safe.
- **Centralised vs region-pinned data:** central is simpler; residency rules may force regionalisation.

STEP 5 – COMMON MISTAKES

- Letting one tenant's raw data influence another's predictions.
- No mechanism to honour deletion across features, models and backups.
- Per-tenant models that don't scale to millions of tenants operationally.
- Ignoring data-residency obligations and storing data in the wrong region.

M22. Design a Cost-Optimisation Strategy for ML Training and Inference at Scale

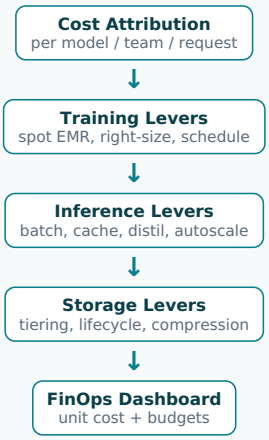
STEP 1 – CLARIFY REQUIREMENTS

Clarify where the spend concentrates (training compute, inference fleet, storage, data movement), the latency/quality SLAs that

constrain savings, and whether the goal is unit-cost (cost per prediction) or total spend. Clarify visibility (do we even attribute cost per model/team?). Confirm fleet-scale millions of inferences and that cost is a stated platform concern at Xero.

STEP 2 – HIGH-LEVEL DESIGN

Attribute cost per model/team first, then attack the biggest levers: spot/transient clusters and right-sizing for training; batching, caching, model distillation/quantisation, and autoscaling for inference; tiered storage and lifecycle policies for data. A FinOps dashboard tracks unit economics.



You can't optimise what you can't measure — attribution comes first, then targeted levers.

STEP 3 – DEEP DIVE

Start with measurement: tag resources so cost is attributable per model and team, exposing the few workloads that dominate spend. For training, transient spot EMR clusters and right-sized instances often cut cost dramatically; schedule off-peak and cache engineered features to avoid recomputation. For inference (usually the bigger long-run cost at millions of users), the highest-leverage moves are precomputing/batching predictions where freshness allows, caching repeated requests, and shrinking models via distillation or quantisation to fit cheaper hardware while holding quality. Autoscale to demand instead of provisioning for peak. Tier and lifecycle storage so cold data moves to cheap classes. Crucially, set guardrails so cost cuts never breach latency or quality SLAs.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Cost vs latency/quality:** smaller models and caching save money but can hurt freshness or accuracy.
- **Spot vs on-demand:** spot is cheap but interruptible — needs checkpointing.
- **Precompute vs real-time:** precompute is cheap but stale; real-time is fresh but costly.
- **Engineering effort vs savings:** some optimisations cost more in eng time than they save.

STEP 5 – COMMON MISTAKES

- Optimising blindly without cost attribution, missing the real hotspots.
- Cutting model size or caching aggressively and silently degrading quality.
- Provisioning the inference fleet for peak 24/7 instead of autoscaling.
- Ignoring storage and data-transfer costs, which quietly accumulate.

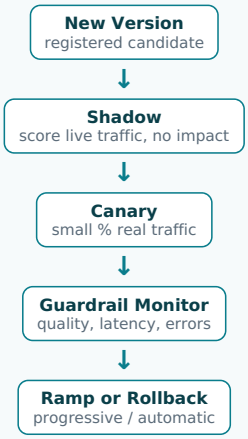
M23. Design a Shadow Deployment and Canary Rollout System for ML Models

STEP 1 – CLARIFY REQUIREMENTS

Clarify the risk we're mitigating (a new model regressing for some users), what 'success' looks like during rollout (quality, latency, error, business guardrails), the rollback trigger and speed, and whether shadow needs to compare predictions without affecting users. Confirm we want this as a reusable platform capability so every model deploys safely.

STEP 2 – HIGH-LEVEL DESIGN

New versions first run in shadow (score live traffic, log predictions, no user impact) for comparison against the champion. If healthy, route a small canary percentage of real traffic, monitor guardrails, then progressively ramp. Automated rollback fires on guardrail breach.



Shadow validates safely; canary limits blast radius; automated rollback bounds damage.

STEP 3 – DEEP DIVE

The point is to bound blast radius and catch regressions before they hit everyone. Shadow mode is uniquely valuable because it compares the candidate against the champion on identical live inputs with zero user risk, surfacing divergence and latency issues offline. Canary then exposes a small, monitored slice to real traffic; tie progression to guardrail metrics (error rate, latency p99, and business KPIs) rather than time alone. Automate rollback so a breach reverts instantly without human latency — the registry's versioning makes this a fast pointer swap. Keep assignment sticky so canary users have a consistent experience. Build this once as a platform so teams inherit safe rollout by default.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Shadow cost vs safety:** shadow doubles inference cost during validation but catches issues risk-free.
- **Fast vs slow ramp:** fast ramp ships value sooner but widens blast radius if something's wrong.
- **Automated vs manual rollback:** automation is fast but can over-trigger on noisy metrics.
- **Per-model vs platform rollout:** a shared platform standardises safety but must handle diverse models.

STEP 5 – COMMON MISTAKES

- Deploying straight to 100% of users with no canary, maximising blast radius.
- Ramping on a timer while ignoring guardrail metrics.
- No automated rollback, so damage continues while humans react.
- Comparing shadow vs champion on different inputs, making the comparison meaningless.

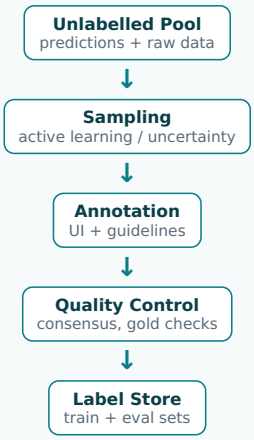
M24. Design a Human-in-the-Loop Labelling / Annotation Pipeline

STEP 1 – CLARIFY REQUIREMENTS

Clarify the labelling need (bootstrapping a new model, correcting low-confidence predictions, building eval sets), label quality requirements, who labels (internal experts, users via product corrections, vendors), throughput, and how labels flow back into training. Clarify cost and the need for quality control (inter-annotator agreement). Confirm financial-domain labels need expertise and auditability.

STEP 2 – HIGH-LEVEL DESIGN

A sampling strategy selects items to label (uncertain or representative ones), routes them to annotators via a labelling UI, applies quality control (consensus, gold questions), and writes validated labels to a store that feeds training and evaluation. Active learning prioritises the most informative items.



Active learning spends scarce labelling budget where it most improves the model.

STEP 3 – DEEP DIVE

Labelling is expensive, so spend the budget wisely: active learning selects the items the model is most uncertain about or that are most representative, yielding far more improvement per label than random sampling. Quality control is non-negotiable for financial data — use multiple annotators with agreement metrics, hidden gold-standard questions to catch bad labellers, and clear guidelines with

adjudication for disputes. Capture in-product user corrections (a near-free, high-quality label source) and merge them with expert labels. Maintain a frozen, clean evaluation set separate from training labels so model comparisons stay honest. Track label provenance for auditability. Feed validated labels into the feature store and retraining loop.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Active learning vs random sampling:** active learning is efficient but can bias the dataset toward hard cases.
- **Expert vs crowd/user labels:** experts are accurate but costly; users are cheap but noisier.
- **Consensus cost vs quality:** multiple annotators improve quality but multiply cost.
- **Labelling speed vs guideline rigour:** looser guidelines are faster but reduce consistency.

STEP 5 – COMMON MISTAKES

- Random sampling that wastes budget on items the model already handles.
- No quality control, so noisy labels silently degrade the model.
- Mixing eval labels into training, inflating reported performance.
- Discarding free in-product user corrections.

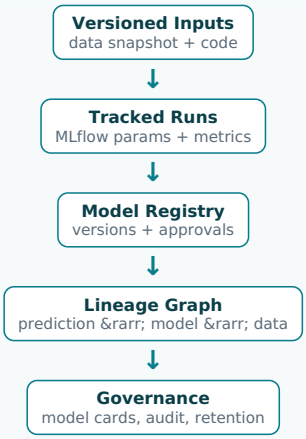
M25. Design a Model Governance, Lineage and Reproducibility System

STEP 1 – CLARIFY REQUIREMENTS

Clarify the obligations (audit, explainability, regulatory compliance for financial decisions), what must be traceable (which model, data, code produced a given prediction), retention requirements, who can approve/deploy models, and how bias/fairness is documented. Confirm this is a cross-cutting platform concern and that fintech demands defensible, reproducible decisions.

STEP 2 – HIGH-LEVEL DESIGN

Every artifact is versioned and linked: data snapshots, code commits, training runs, model versions, and predictions form a connected lineage graph. A governance layer enforces approvals, documents model cards (intended use, metrics, fairness), and retains records for audit. Any prediction can be traced end-to-end.



Reproducibility is a chain: any production prediction traces back to the exact data and code that made it.

STEP 3 – DEEP DIVE

Reproducibility is a chain of immutable links: a prediction points to a model version, which points to a training run, which points to a data snapshot and code commit. If any link is mutable or missing, audits fail. Enforce immutable, versioned data snapshots and pinned environments so a run can be re-executed identically. Layer governance on top: model cards documenting intended use, performance by segment, and fairness assessments; approval gates so high-stakes financial models need sign-off; and retention policies meeting regulatory timelines. Capture lineage automatically as a by-product of the platform, not manual paperwork, or it won't be maintained. This both satisfies regulators and accelerates debugging when a model misbehaves.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Governance rigour vs velocity:** heavy approvals add safety but slow iteration — tier by model risk.
- **Automated vs manual lineage capture:** automated is reliable but needs platform investment; manual rots quickly.
- **Full retention vs storage cost:** regulators want long retention; storage and complexity grow.
- **Centralised governance vs team autonomy:** central ensures compliance; teams want speed.

STEP 5 – COMMON MISTAKES

- Mutable training data or unpinned environments, making runs irreproducible.
- Capturing lineage manually so it drifts out of date and fails audits.
- No approval gates on high-stakes financial models.
- Treating governance as paperwork bolted on at the end rather than built into the platform.

Section B — 20 AI Agent (LLM / RAG) System Design Questions

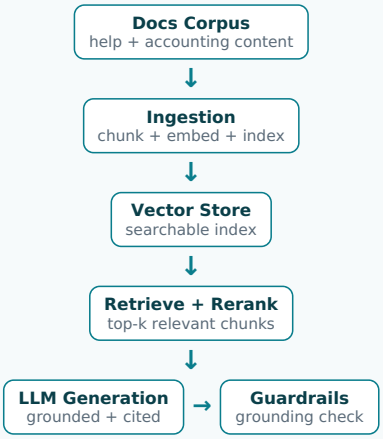
A1. Design a RAG System over Xero Help Docs and Accounting Knowledge

STEP 1 – CLARIFY REQUIREMENTS

Clarify the corpus (help articles, accounting guidance, possibly tenant-specific data) and its update frequency, latency budget for a chat response, accuracy/grounding requirements (must cite sources, must not hallucinate financial advice), languages, and scale (millions of users). Clarify whether answers are read-only Q&A or can trigger actions. Confirm cost per query matters and that wrong answers about finances are high-risk.

STEP 2 – HIGH-LEVEL DESIGN

Documents are chunked, embedded, and indexed in a vector store offline. At query time, the question is embedded, top-k relevant chunks are retrieved (often with reranking), assembled into a grounded prompt, and the LLM generates a cited answer. Guardrails check grounding before returning.



Offline indexing (top path) vs online query (bottom path); citations make answers verifiable.

STEP 3 – DEEP DIVE

Retrieval quality determines answer quality — a perfect LLM can't answer from irrelevant context. Invest in chunking (semantic, overlapping, sized to the model) and a reranker that reorders the vector store's candidates for precision. Force grounding by instructing the model to answer only from retrieved context and to cite chunk sources, then validate the answer is supported before returning, so it can't fabricate financial guidance. Keep the index fresh with an incremental ingestion pipeline triggered on content changes. Cache frequent queries to cut cost and latency. Evaluate retrieval (recall@k) and answer (faithfulness, helpfulness) separately. For ambiguous questions, retrieve broadly and let the model ask a clarifying question rather than guess.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Chunk size:** small chunks are precise but lose context; large chunks keep context but dilute relevance.
- **Retrieval depth (k) vs cost/latency:** more chunks improve recall but inflate prompt size and cost.
- **RAG vs fine-tuning:** RAG keeps knowledge fresh and citable; fine-tuning bakes it in but is stale and opaque.
- **Rerank quality vs latency:** a cross-encoder reranker boosts precision but adds a step.

STEP 5 – COMMON MISTAKES

- Treating RAG as 'just embeddings' and neglecting retrieval/reranking quality.
- Not enforcing grounding/citations, letting the model invent financial advice.
- A stale index that doesn't reflect updated help content.
- Evaluating only the LLM, never measuring retrieval recall.

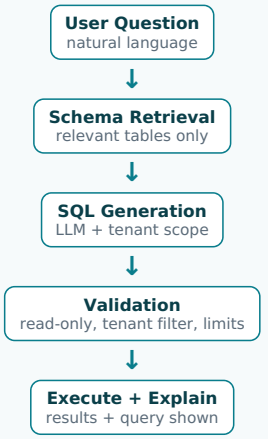
A2. Design a Natural-Language-to-Query (Text-to-SQL) Agent over Financial Data

STEP 1 – CLARIFY REQUIREMENTS

Clarify the scope of questions ('how much did I spend on travel last quarter?'), the underlying schema and multi-tenancy (queries must be scoped to the asking tenant), accuracy and safety (a wrong number about someone's finances is serious), read-only enforcement, latency, and how ambiguous questions are handled. Confirm we must never leak cross-tenant data and must show the query for trust.

STEP 2 – HIGH-LEVEL DESIGN

The agent grounds the LLM with the relevant schema (retrieved per question), generates a SQL query constrained to the tenant, validates and sandboxes it (read-only, row-level tenant filter), executes it, and returns results with the query shown and a natural-language summary.



Safety is enforced after generation: validated, tenant-scoped, read-only SQL before any execution.

STEP 3 – DEEP DIVE

Safety and tenant isolation are the hard requirements. Never trust the LLM's SQL directly: parse and validate it to ensure it is read-only, references only permitted tables, includes the tenant filter (ideally enforced by injecting a row-level security predicate or running under a tenant-scoped role rather than relying on the prompt), and has sensible limits. Ground generation by retrieving only the schema relevant to the question, since dumping the whole schema bloats the prompt and confuses the model. Show the generated query and let users verify, building trust for financial answers. Handle ambiguity by asking a clarifying question. Cache schema embeddings and common queries. Evaluate on a labelled question→SQL set for execution accuracy.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Prompt-injected tenant filter vs DB-enforced isolation:** DB-level (RLS/role) is far safer than trusting the model.
- **Full schema vs retrieved schema:** retrieval scales to large schemas but can miss needed tables.
- **Direct execution vs human confirmation:** auto-run is smooth; confirmation is safer for high-stakes queries.
- **LLM SQL vs templated queries:** templates are safe but rigid; LLM is flexible but riskier.

STEP 5 – COMMON MISTAKES

- Relying on the prompt alone for tenant isolation, risking cross-tenant data leaks.
- Executing unvalidated LLM SQL (write/delete or unbounded scans).
- Stuffing the entire schema into the prompt and degrading accuracy.
- Hiding the query, so users can't verify a financial number.

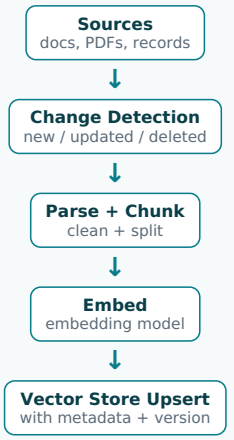
A3. Design a Production RAG Ingestion and Indexing Pipeline

STEP 1 – CLARIFY REQUIREMENTS

Clarify the source variety (docs, PDFs, structured records), update frequency and freshness SLA, deletion/versioning needs (content removed must leave the index), embedding model choice and dimensionality, scale of the corpus, and multi-tenancy. Confirm reprocessing strategy when the embedding model or chunking changes. Confirm cost and that stale or orphaned chunks degrade answers.

STEP 2 – HIGH-LEVEL DESIGN

A pipeline detects changed source content, parses and chunks it, generates embeddings, and upserts them into the vector store with metadata (source, tenant, version). Deletions remove chunks; a versioned index supports re-embedding. Orchestrated incrementally on content changes.



Incremental upserts keep the index fresh; metadata enables tenant scoping and deletion.

STEP 3 – DEEP DIVE

Freshness and consistency are the operational challenges most prototypes ignore. Detect changes incrementally (content hashes or change feeds) so you re-embed only what changed rather than the whole corpus each run. Handle deletions explicitly — removed source content must purge its chunks, or the model will cite deleted material. Store rich metadata per chunk (source ID, tenant, timestamp,

version) to enable tenant-scoped retrieval, filtering, and clean updates. When the embedding model or chunking strategy changes, you must re-embed the entire corpus into a new index version and switch over atomically, so plan for reindexing. Make upserts idempotent and the pipeline restartable. Monitor index size, orphaned chunks, and embedding cost.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Full reindex vs incremental:** incremental is cheap and fast; full reindex is needed when models/chunking change.
- **Embedding quality vs cost/dimension:** larger embeddings improve retrieval but cost more to store and search.
- **Real-time vs batch ingestion:** real-time keeps the index fresh; batch is cheaper and simpler.
- **Chunk granularity:** finer chunks improve precision but multiply storage and index size.

STEP 5 – COMMON MISTAKES

- Re-embedding the entire corpus on every run instead of incrementally.
- Not handling deletions, leaving orphaned chunks that get cited.
- No index versioning, making embedding-model upgrades a risky big-bang.
- Missing tenant metadata, breaking isolation at retrieval time.

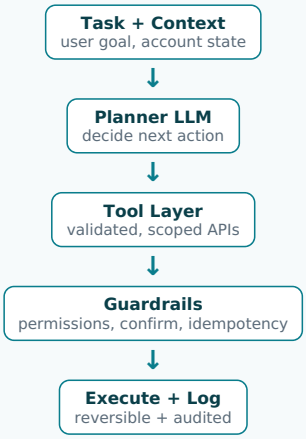
A4. Design an AI Agent That Automates Bookkeeping Tasks (Tool-Using Agent)

STEP 1 – CLARIFY REQUIREMENTS

Clarify the tasks (categorise transactions, match payments to invoices, flag anomalies), the autonomy level (suggest vs auto-execute), which actions are reversible, how the agent calls tools/APIs safely, approval/audit requirements, and failure handling. Confirm the agent operates on real financial records, so safety, idempotency and human oversight are paramount, and that we serve millions of small businesses.

STEP 2 – HIGH-LEVEL DESIGN

An agent loop: a planner LLM decides the next step, calls a constrained set of tools (read transactions, propose categorisation, match invoices) via a validated function-calling interface, observes results, and iterates. High-impact actions require confirmation; everything is logged and reversible.



The tool layer and guardrails — not the LLM — enforce what the agent is actually allowed to do.

STEP 3 – DEEP DIVE

Constrain the agent at the tool boundary, never via prompt alone. Each tool is a narrow, validated, permission-checked API (e.g. 'propose_category', not 'run_arbitrary_update'), and the agent can only do what its tools allow. Distinguish reversible suggestions (safe to auto-apply, e.g. proposing a category the user can undo) from irreversible/high-impact actions (require explicit confirmation). Make every action idempotent and logged so retries don't double-post and an audit trail exists. Bound the agent loop (max steps, timeouts) to avoid runaway cost or loops. Use the LLM for planning and language, but enforce business rules deterministically in the tool layer. Evaluate end-to-end task success and intervention rate, not just LLM output quality.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Autonomy vs control:** more automation reduces toil but increases risk on real financial records.
- **LLM-driven planning vs fixed workflows:** agents handle variety but are less predictable than deterministic flows.
- **Confirm-everything vs auto-execute:** confirmation is safe but adds friction; tune by action reversibility.
- **Tool breadth vs safety:** more tools enable more tasks but widen the attack/error surface.

STEP 5 – COMMON MISTAKES

- Letting the LLM call powerful/unconstrained tools, risking damage to financial records.
- Auto-executing irreversible actions without confirmation.
- Non-idempotent tools that double-post on retry.
- No loop bounds, so the agent loops indefinitely and burns cost.

A5. Design an LLM Evaluation and Guardrails Framework

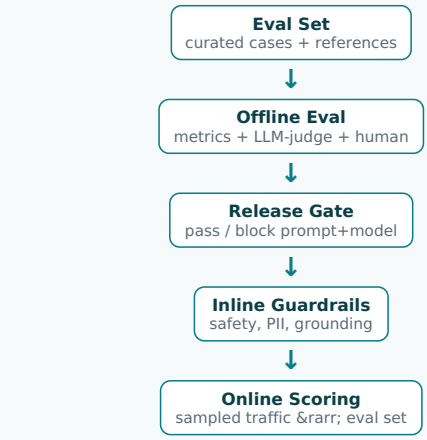
STEP 1 – CLARIFY REQUIREMENTS

Clarify what we evaluate (correctness, grounding/faithfulness, safety, tone, format), offline (pre-deploy) vs online (production)

evaluation, whether we have reference answers or use LLM-as-judge, the latency budget for inline guardrails, and the regulatory bar for financial content. Confirm we need both a pre-release gate and continuous production monitoring, and a way to catch regressions when prompts/models change.

STEP 2 – HIGH-LEVEL DESIGN

Offline: a curated eval set scores candidate prompts/models on multiple dimensions (automatic metrics + LLM-as-judge + human spot-checks) as a release gate. Online: input/output guardrails run inline (safety, PII, grounding), and sampled production traffic is scored for ongoing quality, feeding back into the eval set.



Offline gate prevents regressions; inline guardrails and online scoring catch what tests miss.

STEP 3 – DEEP DIVE

LLM features regress silently when a prompt or model changes, so a curated, versioned eval set is the safety net — run it as a gate before any prompt/model change ships, scoring faithfulness, correctness, safety and format. Combine cheap automatic checks, LLM-as-judge (calibrated against human labels, since judges are biased), and periodic human review. Separate evaluation (does the answer meet the bar?) from guardrails (inline runtime checks that block unsafe/ungrounded/PII-leaking outputs within the latency budget). In production, sample and score real traffic, then feed failures back to grow the eval set — this closes the research/production loop for LLMs, mirroring how ML models are monitored. Track quality, not just latency and cost.

STEP 4 – TRADE-OFFS TO DISCUSS

- **LLM-as-judge vs human eval:** judges scale cheaply but are biased; humans are accurate but slow and costly.
- **Inline guardrail strictness vs latency/UX:** stronger checks add latency and may over-block.
- **Broad eval coverage vs maintenance:** large eval sets catch more but are costly to curate.
- **Reference-based vs reference-free metrics:** references are reliable but expensive to create.

STEP 5 – COMMON MISTAKES

- Shipping prompt/model changes with no eval gate and regressing silently.
- Trusting LLM-as-judge without calibrating it against human labels.
- Confusing offline eval with runtime guardrails — you need both.
- Never feeding production failures back into the eval set.

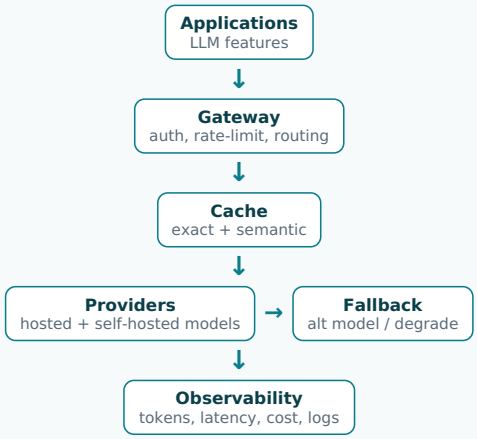
A6. Design an LLM Gateway / Inference Proxy (Routing, Caching, Fallback)

STEP 1 – CLARIFY REQUIREMENTS

Clarify which models/providers we front (hosted APIs, self-hosted), the goals (cost control, reliability, observability, governance), rate-limit and quota needs per team, caching opportunities, and fallback behaviour when a provider is down or slow. Confirm centralisation is desired so every LLM feature shares one controlled, observable entry point rather than each team calling providers directly.

STEP 2 – HIGH-LEVEL DESIGN

A gateway sits between applications and all model providers. It handles auth, rate limiting, routing (by cost/capability), semantic and exact caching, retries and fallback to alternative models, plus centralised logging of tokens, latency, and cost. PII and guardrail hooks run here too.



One controlled entry point for every LLM call: governance, cost control and resilience in one layer.

STEP 3 – DEEP DIVE

Centralisation is the win: routing every LLM call through one gateway gives uniform observability (token/cost/latency per team and feature), governance (rate limits, model allow-lists, PII redaction), and resilience (retries, provider fallback, circuit breakers) without each team reinventing them. Routing can send simple requests to cheap small models and hard ones to large models, controlling cost. Caching — exact for identical prompts, semantic for near-duplicates — cuts both cost and latency significantly for repetitive queries. Fallback keeps features alive when a provider degrades: retry, switch model, or return a graceful default. Because it's a shared dependency, the gateway itself must be highly available and low-overhead, or it becomes the bottleneck.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Centralised gateway vs direct calls:** central gives control but is a shared point of failure to engineer carefully.
- **Semantic cache hit rate vs correctness:** aggressive semantic caching saves cost but may serve subtly wrong cached answers.
- **Cost routing vs quality:** routing to cheaper models saves money but can lower answer quality.
- **Provider lock-in vs abstraction:** abstracting providers adds flexibility but limits provider-specific features.

STEP 5 – COMMON MISTAKES

- Letting teams call providers directly, losing cost visibility and governance.
- A gateway that adds high latency or becomes a single point of failure.
- Over-aggressive semantic caching that returns stale or mismatched answers.
- No fallback, so one provider outage breaks every LLM feature.

A7. Design a Multi-Agent Orchestration System for Complex Accounting Workflows

STEP 1 – CLARIFY REQUIREMENTS

Clarify whether the task genuinely needs multiple specialised agents (e.g. a planner plus retriever, calculator, and writer) or a single agent suffices, how agents communicate and share state, error propagation between agents, cost/latency of multi-step chains, and where humans approve. Confirm the workflows operate on financial data needing correctness and auditability, and beware over-engineering.

STEP 2 – HIGH-LEVEL DESIGN

An orchestrator (planner) decomposes the goal and delegates sub-tasks to specialised worker agents (retrieval, computation, validation, drafting), each with scoped tools. A shared state/memory passes context; a validator agent checks results; high-impact outputs require human approval. The whole run is traced.

Specialised agents under one orchestrator; a validator and tracing keep multi-step runs correct and auditable.

STEP 3 – DEEP DIVE

First justify the complexity: multi-agent systems add latency, cost, and failure modes, so use them only when a single agent with tools genuinely can't handle the task. When justified, give each agent a narrow role and scoped tools, and pass state explicitly through a shared context rather than relying on long conversations that drift. The critical addition for finance is a validator/critic step that cross-

checks computations and grounding before results are accepted, since errors compound across agents. Trace every agent's inputs, outputs, and tool calls for debugging and audit. Bound total steps and cost. Keep humans in the loop at high-impact decision points. Evaluate the whole workflow's success rate, not individual agents.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Multi-agent vs single agent:** multi-agent handles complex decomposition but adds cost, latency and failure surface.
- **Autonomy vs human checkpoints:** fewer checkpoints are faster; more are safer for financial outputs.
- **Shared memory vs message passing:** shared state is simple; explicit messages are cleaner but verbose.
- **Error isolation vs propagation:** one agent's mistake can cascade without a validator.

STEP 5 – COMMON MISTAKES

- Reaching for multi-agent complexity when a single tool-using agent would do.
- No validator, so errors compound silently across agents.
- Unbounded agent chains that explode cost and latency.
- No tracing, making multi-agent failures impossible to debug or audit.

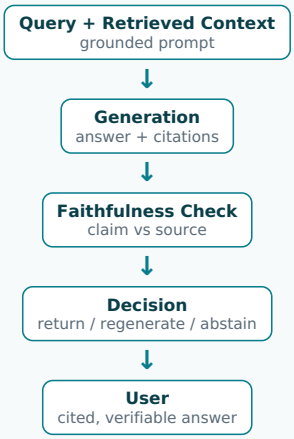
A8. Design a Hallucination Detection and Grounding / Citation System

STEP 1 – CLARIFY REQUIREMENTS

Clarify the risk tolerance (financial answers must be grounded), whether every claim needs a citation, the latency budget for verification, whether we block, flag, or regenerate ungrounded answers, and how we measure faithfulness. Confirm that for accounting guidance a confident wrong answer is worse than an 'I don't know', and that we need citations users can verify.

STEP 2 – HIGH-LEVEL DESIGN

Answers are generated strictly from retrieved context with inline citations. A verification step checks each claim against its cited source (NLI/entailment or LLM-judge); unsupported claims are flagged, regenerated, or the system abstains. Confidence and citations surface to the user.



Generation is grounded by construction; a verification gate catches claims the sources don't support.

STEP 3 – DEEP DIVE

Prevention and detection both matter. Prevent by constraining generation to retrieved context and prompting for citations, so the model has the right material and a habit of grounding. Detect by verifying after generation: for each claim, check whether the cited source actually entails it (an entailment model or a focused LLM-judge), and flag or strip unsupported claims. For finance, prefer abstention ('I couldn't find this in your records/help content') over a confident guess, and make that an acceptable, well-designed outcome. Surface citations users can click to verify. Measure faithfulness on an eval set and monitor abstention and unsupported-claim rates in production. Note verification adds latency and cost, so apply it proportionally to risk.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Verification cost/latency vs safety:** per-claim checking is thorough but slow and expensive.
- **Abstain vs answer:** abstaining avoids wrong answers but can frustrate users expecting help.
- **Strict grounding vs fluency:** forcing citations can make answers stilted or overly hedged.
- **NLI model vs LLM-judge:** NLI is cheap and fast; LLM-judge is flexible but costlier.

STEP 5 – COMMON MISTAKES

- Trusting the model to self-report confidence instead of verifying against sources.
- Never abstaining, so the system always answers even without grounding.
- Citations that don't actually support the claim (citation theatre).
- Verifying nothing in production while assuming offline grounding holds.

A9. Design a Semantic Caching Layer for LLM Responses

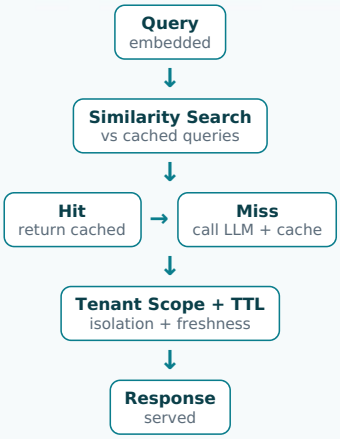
STEP 1 – CLARIFY REQUIREMENTS

Clarify the query distribution (how repetitive are user questions?), correctness sensitivity (a wrong cached financial answer is dangerous), freshness needs (does the underlying data change?), the similarity threshold for a 'hit', and tenant isolation (one tenant's

cached answer must never serve another). Confirm the goal is cost and latency reduction without compromising correctness.

STEP 2 – HIGH-LEVEL DESIGN

Incoming queries are embedded and matched against a cache of prior query embeddings. A close-enough match returns the cached response; otherwise the LLM is called and the new pair is cached. Caching is tenant-scoped and invalidated when underlying data changes.



Semantic match serves near-duplicate questions cheaply; tenant scoping and TTL protect correctness.

STEP 3 – DEEP DIVE

Semantic caching can cut a large fraction of cost and latency because many users ask similar questions, but the similarity threshold is a correctness dial: too loose and it serves a cached answer to a subtly different question (dangerous for finance); too tight and the hit rate collapses. Calibrate it conservatively for high-stakes queries. Crucially, cache must be tenant-scoped — never serve one business's cached answer to another, since the answer may embed their data. Invalidate or TTL entries when the underlying data they depend on changes, or the cache will serve stale financial information. Cache generic/help-content answers aggressively but data-dependent answers cautiously or not at all. Monitor hit rate and any quality impact.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Similarity threshold:** loose = high hit rate but risk of wrong answers; tight = safe but low savings.
- **Cache static vs data-dependent answers:** static help answers cache well; personal financial answers are risky.
- **Cost savings vs freshness:** longer TTLs save more but risk stale data.
- **Exact vs semantic caching:** exact is safe but low hit rate; semantic catches more but risks mismatches.

STEP 5 – COMMON MISTAKES

- A loose threshold that serves answers to the wrong question.
- Cross-tenant cache leaks exposing one business's answer to another.
- No invalidation, serving stale answers after the underlying data changes.
- Caching personalised financial answers as if they were generic.

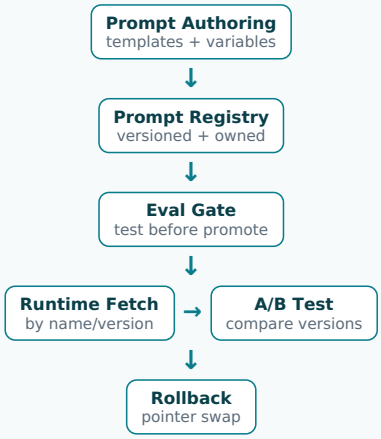
A10. Design a Prompt Management and Versioning System

STEP 1 – CLARIFY REQUIREMENTS

Clarify who edits prompts (engineers, scientists, PMs), the need to version and roll back prompts independently of code, how prompts are tested before release (tie to the eval framework), A/B testing of prompts, and environment separation (dev/prod). Confirm prompts are first-class production assets whose changes can regress quality, and that we serve many LLM features across teams.

STEP 2 – HIGH-LEVEL DESIGN

A prompt registry stores versioned, named prompt templates with metadata and ownership. Applications fetch prompts by name/version at runtime. Changes go through eval gates before promotion; A/B testing routes traffic between prompt versions; rollback is a version pointer change.



Prompts are versioned assets with eval gates and rollback — mirroring the model registry for LLMs.

STEP 3 – DEEP DIVE

Treat prompts like models, not hard-coded strings. A registry decouples prompt changes from code deploys, so non-engineers can iterate and changes are versioned, attributable, and reversible. Every prompt change must pass the eval gate before promotion, because a small wording change can regress quality across millions of requests — this is the LLM analogue of the model promotion pipeline. Support A/B testing so prompt variants are compared on real traffic with metrics, not vibes. Separate environments so experimental prompts don't hit production. Parameterise prompts with typed variables to prevent injection of unescaped user input. Roll back instantly via a version pointer if a prompt misbehaves. Log which prompt version produced each response for debugging and audit.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Registry flexibility vs governance:** easy editing speeds iteration but needs gates to prevent bad changes shipping.
- **Decoupled prompts vs code coupling:** decoupling enables fast iteration but adds a runtime dependency.
- **A/B rigour vs speed:** testing every prompt change is safe but slows iteration.
- **Centralised vs per-team prompts:** central enables reuse/governance; per-team maximises autonomy.

STEP 5 – COMMON MISTAKES

- Hard-coding prompts in code, so changes need a deploy and can't be rolled back fast.
- Shipping prompt changes with no eval gate and regressing quality.
- No record of which prompt version produced an output, breaking debugging.
- Injecting raw user input into prompts without escaping (prompt injection).

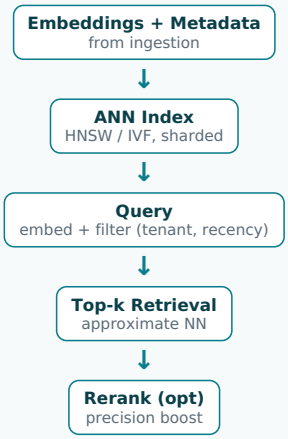
A11. Design a Vector Database / Embedding Store Architecture at Scale

STEP 1 – CLARIFY REQUIREMENTS

Clarify the corpus size (millions to billions of vectors?), query QPS and latency budget, recall requirements (approximate vs exact nearest neighbour), metadata filtering needs (tenant, recency), update frequency (static vs constantly changing), and embedding dimensionality. Confirm multi-tenancy and that retrieval latency is on the critical path of every RAG/agent response.

STEP 2 – HIGH-LEVEL DESIGN

Embeddings are indexed in a vector store using an ANN index (e.g. HNSW/IVF) for fast approximate search, partitioned by tenant, with metadata filters applied at query time. Writes go through the ingestion pipeline; reads serve top-k with optional reranking. The store scales horizontally by sharding.



ANN trades exactness for speed at scale; metadata filtering enforces tenant scope and freshness.

STEP 3 – DEEP DIVE

At scale you must use approximate nearest neighbour, accepting a small recall loss for orders-of-magnitude speed-up; tune the index (HNSW parameters) to balance recall, latency, and memory. Metadata filtering is essential and tricky: tenant isolation and recency filters must apply efficiently alongside vector search, so the store needs first-class filtered-ANN support rather than post-filtering that breaks recall. Shard by tenant or hash for horizontal scale, keeping hot tenants balanced. For frequently updated corpora, choose an index that supports incremental updates without full rebuilds. Decide build vs buy: managed vector DBs reduce ops, while self-hosting offers control and cost predictability. Monitor recall (against an exact baseline), latency, and index memory.

STEP 4 – TRADE-OFFS TO DISCUSS

- **ANN vs exact search:** ANN is fast and scalable but loses some recall; exact is precise but slow at scale.
- **Recall vs latency/memory:** higher-recall index settings cost more latency and RAM.
- **Managed vs self-hosted vector DB:** managed cuts ops; self-hosted gives cost and control.
- **Pre- vs post-filtering metadata:** native filtered-ANN preserves recall; naive post-filter can return too few hits.

STEP 5 – COMMON MISTAKES

- Using exact search and failing to scale, or ANN without measuring recall loss.
- Post-filtering metadata after retrieval and silently dropping relevant results.
- No tenant partitioning, risking cross-tenant retrieval.
- Choosing an index that needs full rebuilds for a frequently-updated corpus.

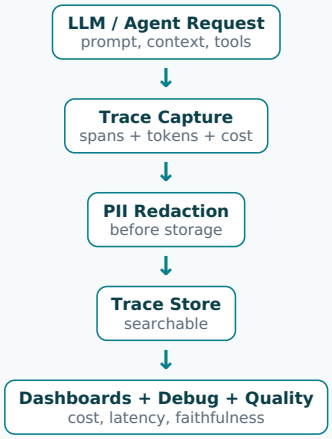
A12. Design an LLM Observability and Tracing System

STEP 1 – CLARIFY REQUIREMENTS

Clarify what we trace (full request: prompt, retrieved context, tool calls, tokens, latency, cost, output, user feedback), the volume and sampling strategy, PII handling in logs, who consumes traces (engineers debugging, FinOps tracking cost, quality teams), and retention. Confirm that without tracing, LLM/agent failures are nearly impossible to debug, and that cost and quality must be observable per feature.

STEP 2 – HIGH-LEVEL DESIGN

Every LLM/agent request emits a structured trace spanning retrieval, model calls, tool calls, and the final output, with tokens, latency, and cost. Traces flow to a store backing dashboards (cost, latency, quality), search/debugging, and sampled quality scoring. PII is redacted before storage.



End-to-end traces make non-deterministic LLM behaviour debuggable and cost attributable.

STEP 3 – DEEP DIVE

LLM systems are non-deterministic and multi-step, so a single response failure could come from bad retrieval, a wrong tool call, or the model itself — only end-to-end tracing reveals which. Capture the full span tree (retrieval results, each model and tool call, intermediate reasoning where available, tokens, latency, cost) linked by a trace ID. Attribute cost and latency per feature and team so spend is visible and controllable. Redact PII before persisting logs, since prompts contain customer financial data. Sample traffic for detailed quality scoring (faithfulness, helpfulness) rather than scoring everything. Surface dashboards for cost/latency/quality trends and an interface to replay and debug a specific failing trace. This is the LLM analogue of model monitoring.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Full tracing vs cost/volume:** tracing everything is complete but expensive; sampling saves cost but can miss rare issues.
- **Detail vs PII risk:** richer logs aid debugging but increase privacy exposure — redact carefully.
- **Real-time vs batch quality scoring:** real-time catches issues fast; batch is cheaper.
- **Retention vs storage cost:** longer retention aids audits but grows cost.

STEP 5 – COMMON MISTAKES

- Logging only the final output, so multi-step failures can't be diagnosed.
- Storing prompts with PII unredacted.
- No cost attribution, so a runaway feature's spend is invisible.
- Tracing without quality scoring, seeing latency but not whether answers are good.

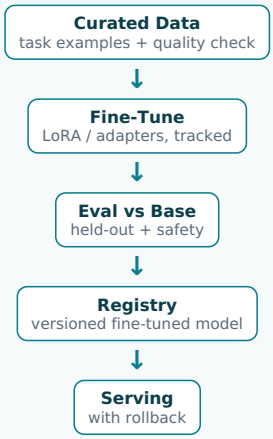
A13. Design a Fine-Tuning / Model-Customisation Pipeline (and When to Use It)

STEP 1 – CLARIFY REQUIREMENTS

Clarify the goal (domain tone/format, a narrow task, cost reduction via a smaller specialised model) and whether RAG or prompting would suffice first, the training data source and quality, evaluation to prove improvement, and how a fine-tuned model is registered, served, and rolled back. Confirm we won't fine-tune to add knowledge (that's RAG's job) and that data privacy applies to training data.

STEP 2 – HIGH-LEVEL DESIGN

A pipeline curates and validates training examples, runs efficient fine-tuning (e.g. LoRA/adapters) tracked in MLflow, evaluates the result against the base model on a held-out set, and registers it for serving with versioning and rollback — mirroring the ML model lifecycle.



Fine-tune for behaviour/format/cost, not knowledge; gate on proven improvement over the base model.

STEP 3 – DEEP DIVE

The first decision is whether to fine-tune at all: use RAG to add knowledge (it's fresh and citable), prompting for simple behaviour changes, and fine-tuning only for consistent task behaviour, output format, tone, or to make a smaller cheaper model match a larger one. Data quality dominates outcomes — a few thousand clean, representative examples beat noisy bulk; validate and de-duplicate, and ensure training data carries no privacy violations. Use parameter-efficient methods (LoRA/adapters) to cut cost and enable many task-specific variants. Treat the fine-tuned model exactly like any production model: track the run, evaluate against the base model on a frozen set (including safety regressions), register, version, and keep rollback. Re-evaluate when base models update.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Fine-tune vs RAG vs prompting:** fine-tune fixes behaviour but is costly and static; RAG/prompting are cheaper and more flexible.
- **Full fine-tune vs LoRA/adapters:** adapters are cheap and modular; full tuning is more powerful but expensive.
- **Small fine-tuned vs large general model:** a tuned small model is cheaper to serve but narrower.
- **Data quantity vs quality:** small clean datasets often beat large noisy ones.

STEP 5 – COMMON MISTAKES

- Fine-tuning to inject knowledge that RAG should supply — expensive and quickly stale.
- Training on noisy or privacy-violating data and degrading or exposing the model.
- No evaluation proving the fine-tune beats the base model.
- No versioning/rollback, so a bad fine-tune is stuck in production.

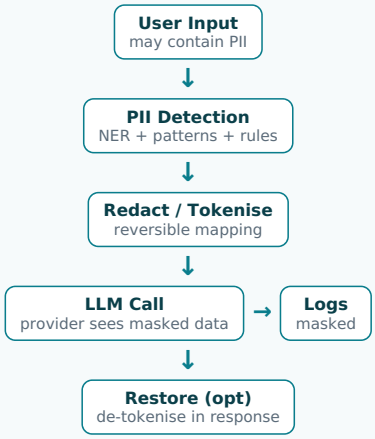
A14. Design a PII Redaction and Data-Privacy Layer for LLM Features

STEP 1 – CLARIFY REQUIREMENTS

Clarify what counts as PII/sensitive financial data in scope, where redaction applies (before sending to external providers, before logging, in outputs), the accuracy bar (missing PII is a leak; over-redaction breaks usefulness), latency budget, and whether redaction must be reversible (to restore entities in the response). Confirm compliance obligations and that customer financial data must never leak to third-party LLMs unprotected.

STEP 2 – HIGH-LEVEL DESIGN

A privacy layer detects and masks PII on the path to the model and to logs, optionally tokenising entities so they can be restored in the response. It sits in the LLM gateway so every feature inherits it, with policies per data type and provider.



Centralised in the gateway; reversible tokenisation lets the model work without ever seeing raw PII.

STEP 3 – DEEP DIVE

Place this in the gateway so protection is universal, not per-team. Detection combines pattern rules (account numbers, emails) with NER and domain rules; tune toward high recall since a miss is a leak, while watching over-redaction that strips needed context. Reversible tokenisation is powerful: replace entities with placeholders before the call, let the model reason over masked text, then restore the real values in the response — the provider never sees raw PII, yet the answer is complete. Redact before logging too, since traces persist.

Apply stricter policies for external providers than self-hosted models. Consider data-residency and contractual controls. Monitor for leaks with detection on outputs as well, and evaluate redaction precision/recall on a labelled set.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Recall vs over-redaction:** aggressive redaction prevents leaks but can strip context the model needs.
- **Reversible tokenisation vs simple masking:** reversible preserves usefulness but adds a mapping to secure.
- **Self-hosted vs external models:** self-hosting avoids sending PII out but costs more to run.
- **Latency vs thoroughness:** deeper detection adds latency on every request.

STEP 5 – COMMON MISTAKES

- Sending raw customer financial data to external LLM providers.
- Redacting inputs but logging the raw prompt with PII.
- Over-redacting so the model loses the context to answer.
- Treating redaction as per-feature instead of a shared gateway capability.

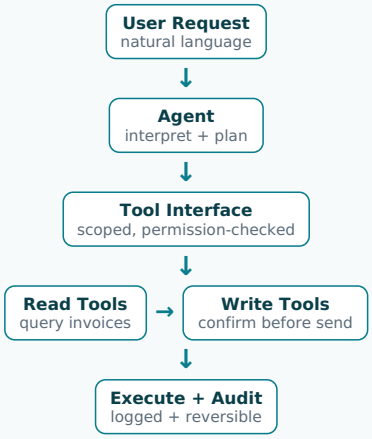
A15. Design an Agentic Invoice-Query Assistant with Safe Tool Calling

STEP 1 – CLARIFY REQUIREMENTS

Clarify the queries ('which invoices are overdue?', 'remind this customer'), which tools are read-only vs state-changing, how tenant scope is enforced on every tool call, confirmation for actions (sending a reminder), and handling of ambiguous requests. Confirm the assistant touches real customer relationships and money, so safety, scoping, and auditability are paramount.

STEP 2 – HIGH-LEVEL DESIGN

An agent interprets the request, calls scoped tools (query invoices, draft reminder) through a validated function-calling interface that enforces tenant scope and read/write permissions, asks for confirmation before state-changing actions, executes, and logs everything. Read queries are answered with data shown.



Every tool call is tenant-scoped and permission-checked; state-changing actions need confirmation.

STEP 3 – DEEP DIVE

The safety model lives in the tool interface, not the prompt. Each tool enforces tenant scope server-side (the agent cannot query another tenant's invoices even if manipulated), checks permissions, and validates arguments. Separate read tools (safe to call freely) from write tools (sending reminders, recording payments) that require explicit user confirmation and are idempotent so retries don't double-send. Handle ambiguity by clarifying rather than guessing which customer or invoice. Log every interpreted intent, tool call, and result for audit, since these actions affect customer relationships. Bound the agent loop and cost. Surface the underlying data so users trust answers. Evaluate end-to-end on real query→action scenarios, tracking incorrect-action and over-clarification rates.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Action autonomy vs confirmation:** auto-actions are smoother; confirmation prevents costly mistakes with customers.
- **Tool breadth vs safety surface:** more tools enable more help but widen risk.
- **Server-enforced vs prompt-enforced scope:** server-side scoping is the only safe option.
- **Clarify vs assume:** clarifying is safe but adds friction; assuming is smooth but risky.

STEP 5 – COMMON MISTAKES

- Enforcing tenant scope in the prompt instead of in the tool layer.
- Sending reminders or changing state without confirmation.
- Non-idempotent write tools that double-send on retry.
- No audit log of actions that affect customers and money.

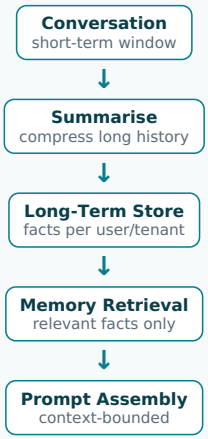
A16. Design a Conversational Assistant with Short-Term and Long-Term Memory

STEP 1 – CLARIFY REQUIREMENTS

Clarify what memory is needed: in-conversation context (short-term) vs persistent user/business facts across sessions (long-term). Clarify privacy and tenant scoping of stored memory, how memory is retrieved into the prompt without bloating it, staleness/correction of stored facts, and the latency budget. Confirm memory contains sensitive financial context that must be isolated and consented.

STEP 2 – HIGH-LEVEL DESIGN

Short-term memory is the managed conversation window (summarised when long). Long-term memory stores salient, consented facts per user/tenant in a retrievable store; relevant memories are fetched and injected into the prompt at query time. Both are tenant-scoped with clear retention and user control.



Short-term = managed window; long-term = retrieved selectively; both strictly tenant-scoped.

STEP 3 – DEEP DIVE

Memory is really context management plus a retrieval problem. Short-term: the conversation grows past the context window, so summarise older turns into a compact running state rather than truncating blindly, preserving intent. Long-term: don't dump everything into the prompt — store salient facts (preferences, recurring entities) and retrieve only those relevant to the current turn, like RAG over the user's own history. Tenant and user scoping is mandatory: memory holds sensitive financial context and must never cross boundaries, with user consent, visibility, correction, and deletion controls. Guard against storing wrong facts that then mislead future answers; allow correction. Bound total injected context to control cost and latency. Evaluate whether memory actually improves task success.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Full history vs summarisation:** full history is faithful but blows the context window and cost; summaries are compact but lossy.
- **Store-everything vs salient memory:** storing all is simple but noisy and costly to retrieve usefully.
- **Memory richness vs privacy:** more stored context helps but increases sensitive-data exposure.
- **Automatic vs user-controlled memory:** automatic is seamless; user control is more trustworthy.

STEP 5 – COMMON MISTAKES

- Stuffing entire history into every prompt, exploding cost and latency.
- Cross-tenant or cross-user memory leakage of financial context.
- Storing incorrect 'facts' that then poison future responses with no correction path.
- No user visibility, consent, or deletion controls over stored memory.

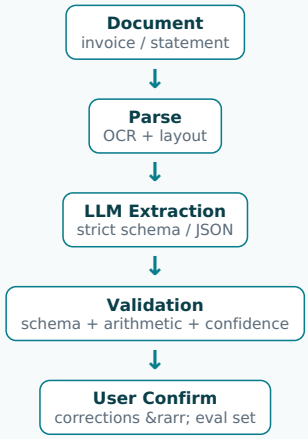
A17. Design a Document-Extraction Agent with Structured-Output Validation

STEP 1 – CLARIFY REQUIREMENTS

Clarify the documents (invoices, contracts, statements) and target schema, the accuracy bar (extracted values post into accounting records), how confidence and validation work, the latency mode (on-upload vs batch), and the human-correction path. Confirm extracted data is financial and errors are costly, distinguishing this LLM-based approach from a pure OCR pipeline by the reasoning over messy layouts.

STEP 2 – HIGH-LEVEL DESIGN

The document is parsed (OCR/layout), an LLM extracts fields into a strict schema (function-calling/JSON mode), outputs are validated against the schema and business rules (totals reconcile), low-confidence or failing fields route to user confirmation, and corrections become training/eval data.



Strict schema + business-rule validation turn free-form LLM output into trustworthy structured data.

STEP 3 – DEEP DIVE

Force structured output and never trust it blindly. Use schema-constrained generation (JSON mode/function calling) so the model returns typed fields, then validate hard: schema conformance, type/format checks, and domain rules (line items sum to the total, tax matches the rate, dates are plausible). Validation catches the LLM's confident mistakes cheaply before they corrupt the ledger. Gate on confidence and validation: auto-fill clean high-confidence fields, surface the rest for quick user confirmation. The correction loop builds a labelled eval set to track extraction accuracy over time and catch regressions when the model or prompt changes. Handle the long tail (odd layouts, languages) and monitor field-level accuracy, not just whole-document success.

STEP 4 – TRADE-OFFS TO DISCUSS

- **LLM extraction vs traditional IDP:** LLMs reason over messy/novel layouts but cost more and can hallucinate fields.
- **Auto-fill vs confirm:** auto-fill reduces toil; confirmation protects the ledger.
- **Strict schema vs flexibility:** strict schemas validate well but struggle with unexpected fields.
- **Per-field confidence vs document-level:** field-level gating is precise but more complex.

STEP 5 – COMMON MISTAKES

- Accepting LLM output without schema and arithmetic validation.
- Auto-posting low-confidence extractions into financial records.
- Free-text output that's hard to validate instead of constrained JSON.
- Not capturing corrections, so extraction accuracy never improves or is even measured.

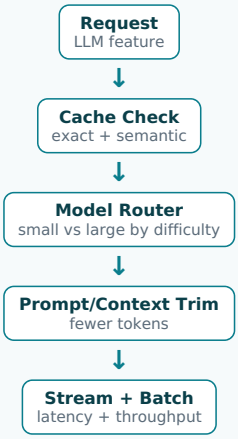
A18. Design a Cost and Latency Optimisation Strategy for LLM Features

STEP 1 – CLARIFY REQUIREMENTS

Clarify where cost/latency concentrate (model size, token volume, retrieval, multi-step agents), the quality floor we must hold, traffic patterns (repetitive vs unique), and whether we control the models (self-hosted vs API). Confirm millions of users make per-request cost material and that latency affects UX, and that we must not trade away correctness on financial answers.

STEP 2 – HIGH-LEVEL DESIGN

Layered levers: route easy requests to small/cheap models and hard ones to large models; cache (exact + semantic); trim prompts and retrieved context; stream responses for perceived latency; batch where possible; and consider distilling/self-hosting high-volume features. A gateway centralises and measures these.



Stack cheap wins (cache, routing, trimming) before expensive ones (distillation/self-hosting); never below the quality floor.

STEP 3 – DEEP DIVE

Order the levers by effort and risk. Cheapest first: cache repetitive queries (exact and, carefully, semantic) and trim bloated prompts and over-retrieved context, since tokens drive both cost and latency. Then route by difficulty — a small model handles most requests and escalates only hard ones to a large model, often cutting cost dramatically while holding quality if you measure it. Stream tokens to improve perceived latency even when total time is unchanged. Batch independent requests for throughput. For very high-volume

features, distilling to a smaller fine-tuned model or self-hosting can beat per-token API pricing. Critically, attach a quality gate to every optimisation so cost cuts never silently degrade financial answers; measure quality, cost, and latency together.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Model routing:** cheap models save cost but can lower quality — needs a confidence/quality check to escalate.
- **Semantic caching:** big savings but risk of serving a near-miss answer.
- **Context trimming:** fewer tokens cut cost but may drop needed information.
- **Self-host vs API:** self-hosting cuts unit cost at high volume but adds operational burden.

STEP 5 – COMMON MISTAKES

- Cutting cost (smaller model, less context) without measuring the quality hit.
- Optimising token count while ignoring the dominant cost (e.g. an over-eager agent loop).
- Aggressive semantic caching that serves subtly wrong answers.
- Treating latency as total time only, ignoring streaming for perceived speed.

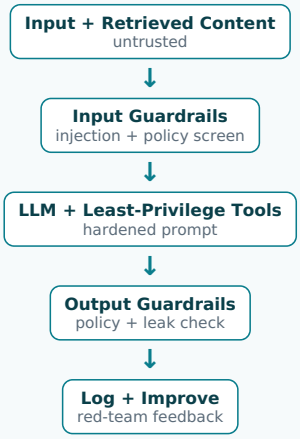
A19. Design an LLM Safety, Moderation and Jailbreak-Defence Pipeline

STEP 1 – CLARIFY REQUIREMENTS

Clarify the threats (harmful/inappropriate content, prompt injection via documents/tools, jailbreaks, data exfiltration, leaking system prompts), where checks run (input and output), the latency budget for inline moderation, and the action on detection (block, sanitise, regenerate). Confirm a financial product's reputational and compliance stakes, and that retrieved/user content is untrusted.

STEP 2 – HIGH-LEVEL DESIGN

Input guardrails screen and sanitise user input and any retrieved/tool content for injection and policy violations; the model runs with a hardened system prompt and least-privilege tools; output guardrails screen responses before they reach the user. Detections are logged and feed defences.



Treat all external content as untrusted; defend at input and output, and constrain tool privileges.

STEP 3 – DEEP DIVE

Defence in depth, with prompt injection as the headline risk for agents: any retrieved document or tool result can contain instructions that hijack the agent, so treat all non-system content as untrusted data, never instructions, and screen it before it enters the prompt. Constrain tools to least privilege so even a successful injection can't do real damage (it can't call a powerful tool it was never granted). Harden the system prompt and avoid exposing secrets to it. Output guardrails catch harmful content and accidental leakage of system prompts or other tenants' data before the user sees them. Log detections and run regular red-teaming to find new bypasses, feeding fixes back — security is continuous, not one-time. Balance strictness against false positives that block legitimate financial questions.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Strict moderation vs false positives:** aggressive filtering blocks attacks but can refuse legitimate queries.
- **Inline checks vs latency:** thorough input/output screening adds latency to every request.
- **Tool capability vs safety:** powerful tools enable more but raise injection blast radius.
- **In-house vs third-party moderation:** third-party is quick to adopt; in-house fits the domain.

STEP 5 – COMMON MISTAKES

- Treating retrieved/tool content as trusted instructions, enabling prompt injection.
- Granting agents broad tool privileges, so an injection causes real harm.
- Screening input but not output, leaking system prompts or other tenants' data.
- Treating safety as a one-off instead of continuous red-teaming.

A20. Design an Offline-to-Online LLM Evaluation and Continuous-Improvement Loop

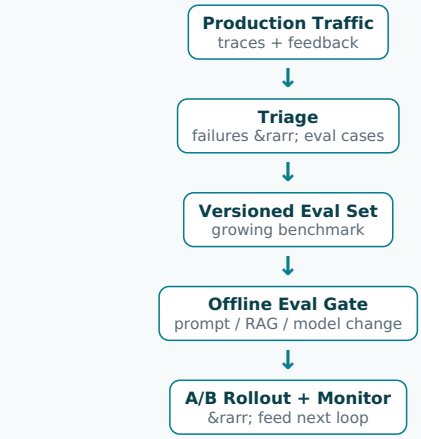
STEP 1 – CLARIFY REQUIREMENTS

Clarify how we capture production signal (user feedback, corrections, abstentions, failures), how it becomes evaluation data, how improvements (prompt, RAG, model changes) are validated before release, and the cadence of iteration. Confirm this is the research/production boundary for LLM features — the analogue of model monitoring + retraining — and that financial-answer quality

must improve safely over time.

STEP 2 – HIGH-LEVEL DESIGN

Production traffic is traced and sampled; failures and user feedback are triaged into a growing, versioned eval set. Proposed changes (prompts, retrieval, models) are evaluated offline against this set as a release gate, rolled out via A/B, monitored online, and their outcomes feed the next iteration.



A closed loop: production failures become eval cases; nothing ships without beating the benchmark.

STEP 3 – DEEP DIVE

This loop is what makes an LLM feature improve safely rather than regress on every change. The eval set is the asset: seed it with representative cases and continuously grow it from real production failures, user corrections, and edge cases, versioning it so results are comparable over time. Any change — prompt tweak, retrieval improvement, model upgrade — must beat the current system on this benchmark before shipping, exactly mirroring champion/challenger for ML models. Use offline eval (automatic metrics, calibrated LLM-judge, human spot-checks) as the gate, then A/B online to confirm real-world lift, monitoring quality, cost, and latency. Feed online results back to refine the eval set. This closes the research→production loop the Xero role centres on, and turns ad-hoc prompt tinkering into disciplined, measurable iteration.

STEP 4 – TRADE-OFFS TO DISCUSS

- **Eval-set growth vs maintenance:** a richer benchmark catches more regressions but costs effort to curate and keep clean.
- **Offline gate strictness vs velocity:** strict gates prevent regressions but slow iteration.
- **LLM-judge vs human eval:** judges scale but need calibration; humans are accurate but slow.
- **Fast iteration vs safety:** shipping quickly delights but risks regressions without the gate.

STEP 5 – COMMON MISTAKES

- Changing prompts/models with no eval gate and regressing silently.
- Never feeding production failures back, so the eval set stagnates.
- Mixing eval cases into few-shot examples, contaminating the benchmark.
- Trusting offline eval alone and skipping online A/B confirmation.

Whiteboard Cheat-Sheet — Drive the Session

> If your mind goes blank at the whiteboard, run this loop out loud. It works for every question in this pack.

Open (30s)	Restate the problem in one sentence. Name the user (small-business owner / accountant) and the business outcome. Ask: real-time or batch? scale? accuracy vs cost vs latency priority?
Frame the seam	Say explicitly: “I’ll separate the research plane from the production plane and make the model registry the contract between them.” This is the single highest-signal sentence you can say in a Xero interview.
Draw 6–8 boxes	Sources → ingestion → processing → feature/index store → train/serve → consumer, with monitoring underneath. Label the arrows with what flows (events, features, predictions).
Go deep on 2–3	Pick the components the interviewer leans on. Show failure modes and how you detect them. Offline↔online parity, idempotent retries, schema/contract checks.
Name trade-offs	Batch vs stream, build vs buy, global vs per-tenant model, fine-tune vs RAG, sync vs async. State what you would pick and why for *this* scale.
Close like an owner	Cost at fleet scale, rollout (shadow → canary), monitoring/alerting, and how you’d document the interface and lift the team. That is the Staff signal.

Twelve Reusable Patterns to Name by Vocabulary

- **Offline/online feature parity** — same transformation code path for training and serving to kill training/serving skew.
- **Model registry as contract** — versioned, signed artifact with metadata; the only handoff between research and prod.

- **Shadow & canary** — mirror live traffic, compare, then ramp by percentage with auto-rollback on guardrail breach.
- **Idempotent, backfillable pipelines** — partitioned by date, safe to re-run, lineage tracked in the orchestrator.
- **Online + offline store split** — low-latency KV for serving, columnar lake for training; one source of truth.
- **Drift & quality monitoring** — input drift, prediction drift, delayed-label metrics; alert before users notice.
- **Human-in-the-loop** — route low-confidence cases to review; feed corrections back as labels.
- **Multi-tenant isolation** — tenant-scoped data access, per-tenant filtering in retrieval, no cross-tenant leakage.
- **RAG grounding + citations** — retrieve, ground, cite, and refuse when context is insufficient.
- **LLM gateway** — one proxy for routing, caching, rate-limits, fallback, cost tracking and guardrails.
- **Eval harness** — offline golden sets + online metrics; gate every prompt/model change through it.
- **Graceful degradation** — cache, smaller fallback model, or rules when the primary path is slow or down.

Final Reminders

- Think out loud. The interview measures reasoning, not silence.
- Always state scale and latency assumptions before drawing.
- Make the research→production interface explicit in every answer.
- Treat privacy, multi-tenancy and auditability as first-class for financial data.
- End each answer with cost, rollout and monitoring — production readiness is the theme.
- When unsure, fall back to the reference skeleton and specialise it.